

SBCL MANUAL

Contents

1	Getting Support and Reporting Bugs	3
1.1	Volunteer Support	3
1.2	Commercial Support	4
1.3	Reporting Bugs	4
1.3.1	How to Report Bugs Effectively	4
1.3.2	How to Report Signal-related Bugs	5
2	Introduction	5
2.1	ANSI Conformance	5
2.2	Extensions	6
2.3	Idiosyncrasies	7
2.3.1	Declarations	7
2.3.2	FASL format	8
2.3.3	Compiler-only Implementation	8
2.3.4	Defining Constants	8
2.3.5	Style Warnings	9
2.4	Development Tools	9
2.4.1	Editor Integration	9
2.4.2	Language Reference	9
2.4.3	Generating Executables	10
2.5	More SBCL Information	10
2.5.1	SBCL Homepage	10
2.5.2	Online Documentation	10
2.5.3	Additional Documentation Files	10
2.5.4	Internals Documentation	10
2.6	More Common Lisp Information	11
2.6.1	Internet Community	11
2.6.2	Third-party Libraries	11
2.6.3	Common Lisp Books	11
2.7	History and Implementation of SBCL	12
3	Starting and Stopping	13
3.1	Starting SBCL	13
3.1.1	Running from Shell	13
3.1.2	Running from Emacs	14
3.1.3	Shebang Scripts	14
3.2	Stopping SBCL	14

3.2.1	Exit	14
3.2.2	End of File	15
3.2.3	Saving a Core Image	15
3.2.4	Exit on Errors	18
3.3	Command Line Options	18
3.3.1	Runtime Options	19
3.3.2	Toplevel Options	20
3.4	Initialization Files	21
3.5	Initialization and Exit Hooks	22
4	Compiler	22
4.1	Diagnostic Messages	22
4.1.1	Controlling Verbosity	22
4.1.2	Diagnostic Severity	23
4.1.3	Understanding Compiler Diagnostics	24
4.2	Handling of Types	27
4.2.1	Declarations as Assertions	28
4.2.2	Precise Type Checking	28
4.2.3	Getting Existing Programs to Run	29
4.2.4	Implementation Limitations	31
4.3	Compiler Policy	31
4.4	Compiler Errors	33
4.4.1	Type Errors at Compile Time	33
4.4.2	Errors During Macroexpansion	34
4.4.3	Read Errors	34
4.5	Open Coding and Inline Expansion	34
4.6	Interpreter	35
4.7	Advanced Compiler Use and Efficiency Hints	35
5	Debugger	36
5.1	Debugger Entry	36
5.1.1	Debugger Banner	36
5.1.2	Debugger Invocation	36
5.2	Debugger Command Loop	37
5.3	Stack Frames	37
5.3.1	Stack Motion	38
5.3.2	How Arguments are Printed	38
5.3.3	Function Names	39
5.3.4	Debug Tail Recursion	39
5.3.5	Unknown Locations and Interrupts	40
5.4	Variable Access	41
5.4.1	Variable Value Availability	41
5.4.2	Note On Lexical Variable Access	43
5.5	Source Location Printing	43
5.5.1	How the Source is Found	44
5.5.2	Source Location Availability	45
5.6	Debugger Policy Control	45

5.7	Exiting Commands	46
5.8	Information Commands	47
5.9	Breakpoint Commands	47
5.9.1	Breakpoint Example	48
5.10	Function Tracing	49
5.11	Single Stepping	52
5.12	Enabling and Disabling the Debugger	52
6	Efficiency	53
6.1	Slot access	53
6.1.1	Structure object slot access	53
6.1.2	Standard object slot access	53
6.2	Stack allocation	53
6.3	Modular arithmetic	57
6.3.1	Signed modular arithmetic	57
6.4	Recognized idioms	57
6.4.1	Count trailing zeros	57
6.5	Global and Always-Bound variables	58
6.6	Miscellaneous Efficiency Issues	58

[in package SB-MANUAL]

This manual, for SBCL version 2.6.5.15.pax-doc.3-8fa8cc6db-WIP, generated on 2026-06-03, is part of the SBCL software system. See the `readme` file for more information.

This manual is largely derived from the manual for the CMUCL system, which was produced at Carnegie Mellon University and later released into the public domain. This manual is in the public domain and is provided with absolutely no warranty. See the `copying` and `credits` files for more information.

1 Getting Support and Reporting Bugs

1.1 Volunteer Support

Your primary source of SBCL support should probably be the mailing list `sbcl-help`: in addition to other users SBCL developers monitor this list and are available for advice. As an anti-spam measure subscription is required for posting:

<https://lists.sourceforge.net/lists/listinfo/sbcl-help>

Remember that the people answering your question are volunteers, so you stand a much better chance of getting a good answer if you ask a good question.

Before sending mail, check the list archives at either

http://sourceforge.net/mailarchive/forum.php?forum_name=sbcl-help

or

<http://news.gmane.org/gmane.lisp.steel-bank.general>

to see if your question has been answered already. Checking the bug database is also worth it (see [Reporting Bugs](#)), to see if the issue is already known.

For general advice on asking good questions, see

<http://www.catb.org/~esr/faqs/smart-questions.html>.

1.2 Commercial Support

There is no formal organization developing SBCL, but if you need a paid support arrangement or custom SBCL development, we maintain the list of companies and consultants below. Use it to identify service providers with appropriate skills and interests, and contact them directly.

The SBCL project cannot verify the accuracy of the information or the competence of the people listed, and they have provided their own blurbs below: you must make your own judgement of suitability from the available information - refer to the links they provide, the CREDITS file, mailing list archives, CVS commit messages, and so on. Please feel free to ask for advice on the sbcl-help list.

(At present, no companies or consultants wish to advertise paid support or custom SBCL development in this manual).

1.3 Reporting Bugs

SBCL uses Launchpad to track bugs. The bug database is available at

<https://bugs.launchpad.net/sbcl>

Reporting bugs there requires registering at Launchpad. However, bugs can also be reported on the mailing list `sbcl-bugs`, which is moderated but does *not* require subscribing.

Simply send email to `sbcl-bugs@lists.sourceforge.net` and the bug will be checked and added to Launchpad by SBCL maintainers.

1.3.1 How to Report Bugs Effectively

Please include enough information in a bug report that someone reading it can reproduce the problem, i.e. don't write

```
Subject: apparent bug in PRINT-OBJECT (or *PRINT-LENGTH*?)
PRINT-OBJECT doesn't seem to work with *PRINT-LENGTH*. Is this a bug?
```

but instead

```
Subject: apparent bug in PRINT-OBJECT (or *PRINT-LENGTH*?)
In sbcl-1.2.3 running under OpenBSD 4.5 on my Alpha box, when
I compile and load the file
  (DEFSTRUCT (FOO (:PRINT-OBJECT (LAMBDA (X Y)
                                   (LET ((*PRINT-LENGTH* 4))
                                       (PRINT X Y))))))
      X Y)
then at the command line type
(MAKE-FOO)
```

the program loops endlessly instead of printing the object.

A more in-depth discussion on reporting bugs effectively can be found at

<http://www.chiark.greenend.org.uk/~sgtatham/bugs.html>.

1.3.2 How to Report Signal-related Bugs

If you run into a signal related bug, you are getting fatal errors such as `signal N is [un]blocked` or just hangs, and you want to send a useful bug report then:

- Compile SBCL with `ldb` enabled (feature `:sb-ldb`, see `base-target-features.lisp-expr`).
- Isolate a smallish test case, run it.
- If it just hangs kill it with `sigabrt: kill -ABRT <pidof sbcl>`.
- Print the backtrace from `ldb` by typing `ba`.
- Attach `gdb`: `gdb -p <pidof sbcl>` and get backtraces for all threads: `thread apply all ba`.
- If multiple threads are in play then still in `gdb`, try to get Lisp backtrace for all threads: `thread apply all call backtrace_from_fp($ebp, 100, 0)`. Substitute `$ebp` with `$rbp` on x86-64. The backtraces will appear in the stdout of the SBCL process.
- Send a report with the backtraces and the output (both stdout and stderr) produced by SBCL.
- Don't forget to include OS and SBCL version.
- If available, include information on outcome of the same test with other versions of SBCL, OS, ...

2 Introduction

SBCL is a mostly-conforming implementation of the ANSI Common Lisp standard. This manual focuses on behavior which is specific to SBCL, not on behavior which is common to all implementations of ANSI Common Lisp.

2.1 ANSI Conformance

Essentially every type of non-conformance is considered a bug. (The exceptions involve internal inconsistencies in the standard.) See [Reporting Bugs](#).

- `prog2` returns the primary value of its second form, as specified in the *Arguments and Values* section of the specification for that operator, not that of its first form, as specified in the *Description*.
- The `string` type is considered to be the union of all types `(array c (size))` for all non-`nil` subtypes `c` of `character(0 1)`, excluding arrays specialized to the empty type.

- The `:order` long form option in `define-method-combination` method group specifiers accepts the value `nil` as well as `:most-specific-first` and `:most-specific-last`, in order to allow programmers to declare that the order of methods playing that role in the method combination does not matter.

2.2 Extensions

SBCL comes with numerous extensions, some in core and some in modules loadable with `require`. Unfortunately, not all of these extensions have proper documentation yet.

- **System Definition Tool:** ASDF is a flexible and popular protocol-oriented system definition tool by Daniel Barlow.
- **Foreign Function Interface:** The `sb-alien` package allows interfacing with C-code, loading shared object files, etc. See

1.

`@SB-GROVEL` can be used to partially automate generation of foreign function interface definitions.

- **Recursive Event Loop:** SBCL provides a recursive event loop (`serve-event`) for doing non-blocking IO on multiple streams without using threads.
- **Timeouts and Deadlines:** SBCL allows restricting the execution time of individual operations or parts of a computation using `:timeout` arguments to certain blocking operations, synchronous timeouts and asynchronous timeouts. The latter two affect operations without explicit timeout support (such as standard functions and macros). See `@TIMEOUTS-AND-DEADLINES`.
- **Metaobject Protocol:** The `sb-mop` package provides an implementation of the metaobject protocol for the Common Lisp Object System as described in *The Art of the Metaobject Protocol* by Kiczales et al.
- **Extensible Sequences:** SBCL allows users to define subclasses of the `sequence` class. See `@EXTENSIBLE-SEQUENCES`.
- **Native Threads:** SBCL has native threads on numerous platforms, capable of taking advantage of SMP on multiprocessor machines. See

2.

- **Network Interface:** The `sb-bsd-sockets` module is a low-level networking interface, providing both TCP and UDP sockets. See

3.

- **Introspective Facilities:** The `@SB-INTROSPECT` module offers numerous introspective extensions, including access to function lambda-lists and a cross referencing facility.
- **Operating System Interface:** The `sb-ext` package contains a number of functions for running external processes, accessing environment variables, etc.

The `@SB-POSIX` module provides a lispy interface to standard POSIX facilities.

- **Extensible Streams:** The package `sb-gray` provides an implementation of `@GRAY-STREAMS`.

The `SB-SIMPLE-STREAMS` module is an implementation of the `@SIMPLE-STREAMS` API proposed by Franz Inc.

- **Profiling:** The `sb-profile` package provides an exact, per-function `@DETERMINISTIC-PROFILER`.

The `sb-sprof` module is SBCL's `@STATISTICAL-PROFILER`, capable of call-graph generation and instruction level profiling, which also supports allocation profiling.

- **Customization Hooks:** SBCL contains a number of extra-standard customization hooks that can be used to tweak the behaviour of the system. See `@CUSTOMIZATION-HOOKS-FOR-USERS`.
- **sb-aclepl:** The `@SB-ACLREPL` module provides an Allegro-style toplevel for SBCL, as an alternative to the classic CMUCL-style one.
- **CLTL2 Compatibility Layer:** The `SB-CLTL2` module provides `sb-cltl2:compiler-let` and environment access functionality described in *Common Lisp The Language, 2nd Edition* which were removed from the language during the ANSI standardization process.
- **Executable Delivery:** The `:executable` argument to `sb-ext:save-lisp-and-die` can produce a "standalone" executable containing both an image of the current Lisp session and an SBCL runtime.
- **Bitwise Rotation:** The `@sb-rotate-byte` module provides an efficient primitive for bitwise rotation of integers, an operation required by e.g. numerous cryptographic algorithms but not available as a primitive in ANSI Common Lisp.
- **Test Harness:** The `sb-rt` module is a simple yet attractive regression and unit-test framework.
- **MD5 Sums:** The `@SB-MD5` module provides an implementation of the MD5 message digest algorithm for Common Lisp, using the modular arithmetic optimizations provided by SBCL.

2.3 Idiosyncrasies

The information in this section describes some of the ways that SBCL deals with choices that the ANSI standard leaves to the implementation.

2.3.1 Declarations

Declarations are generally treated as assertions. This general principle, and its implications, and the bugs which still keep the compiler from quite satisfying this principle, are discussed in [Declarations as Assertions](#).

2.3.2 FASL format

SBCL fasl-format is binary compatible only with the exact SBCL version it was generated with. While this is obviously suboptimal, it has proven more robust than trying to maintain fasl compatibility across versions: accidentally breaking things is far too easy, and can lead to hard to diagnose bugs.

The following snippet handles fasl recompilation automatically for ASDF-based systems, and makes a good candidate for inclusion in the user or system initialization file (see [Initialization Files](#)).

```
(require :asdf)

;;; If a fasl was stale, try to recompile and load (once).
(defmethod asdf:perform :around ((o asdf:load-op)
                                   (c asdf:cl-source-file))
  (handler-case (call-next-method o c)
    ;; If a fasl was stale, try to recompile and load (once).
    (sb-ext:invalid-fasl ()
      (asdf:perform (make-instance 'asdf:compile-op) c)
      (call-next-method))))
```

2.3.3 Compiler-only Implementation

SBCL is essentially a compiler-only implementation of Common Lisp. That is, for all but a few special cases, `eval` creates a lambda expression, calls `compile` on the lambda expression to create a compiled function, and then calls `funcall` on the resulting function object. A more traditional interpreter is also available on default builds; it is usually only called internally. This is explicitly allowed by the ANSI standard but leads to some oddities; e.g. at default settings, `functionp` and `compiled-function-p` are equivalent, and they collapse into the same function when SBCL is built without the interpreter.

2.3.4 Defining Constants

SBCL is quite strict about ANSI's definition of `defconstant`. ANSI says that doing `defconstant` of the same symbol more than once is undefined unless the new value is `eq` (0 1) to the old value. Conforming to this specification is a nuisance when the "constant" value is only constant under some weaker test like `string=` or `equal`.

It's especially annoying because, in SBCL, `defconstant` takes effect not only at load time but also at compile time, so that just compiling and loading reasonable code like

```
(defconstant +foobyte+ '(1 4))
```

runs into this undefined behavior. Many implementations of Common Lisp try to help the programmer around this annoyance by silently accepting the undefined code and trying to do what the programmer probably meant.

SBCL instead treats the undefined behavior as an error. Often such code can be rewritten in portable ANSI Common Lisp which has the desired behavior. E.g., the code above can be given

an exactly defined meaning by replacing `defconstant` either with `defparameter` or with a customized macro which does the right thing, e.g.

```
(defmacro define-constant (name value &optional doc)
  `(defconstant ,name (if (boundp ',name) (symbol-value ',name) ,value)
    ,@(when doc (list doc))))
```

or possibly along the lines of the `sb-int:defconstant-eqx` macro used internally in the implementation of SBCL itself. In circumstances where this is not appropriate, the programmer can handle the condition type `sb-ext:defconstant-uneql` and choose either the `continue` restart or `abort` restart as appropriate.

2.3.5 Style Warnings

SBCL gives style warnings about various kinds of perfectly legal code, e.g.

- multiple `defuns` of the same symbol in different units;
- special variables not named in the conventional `*foo*` style, and lexical variables unconventionally named in the `*foo*` style.

This causes friction with people who point out that other ways of organizing code (especially avoiding the use of `defgeneric`) are just as aesthetically stylish. However, these warnings should be read not as *warning, bad aesthetics detected, you have no style* but as *warning, this style keeps the compiler from understanding the code as well as you might like*. That is, unless the compiler warns about such conditions, there's no way for the compiler to warn about some programming errors which would otherwise be easy to overlook. (Related bug: The warning about multiple `defuns` is pointlessly annoying when you compile and then load a function containing `defun` wrapped in `eval-when`, and ideally should be suppressed in that case, but still isn't as of SBCL 0.7.6.)

2.4 Development Tools

2.4.1 Editor Integration

Though SBCL can be used running "bare", the recommended mode of development is with an editor connected to SBCL, supporting not only basic lisp editing (paren-matching, etc), but providing among other features an integrated debugger, interactive compilation, and automated documentation lookup.

Currently *SLIME* (Superior Lisp Interaction Mode for Emacs) together with Emacs is recommended for use with SBCL, though other options exist as well. Historically, the *ILISP* package at <http://ilisp.cons.org/> provided similar functionality, but it does not support modern SBCL versions.

SLIME can be downloaded from <https://slime.common-lisp.dev/>.

2.4.2 Language Reference

CLHS (Common Lisp Hyperspec) is a hypertext version of the ANSI standard, made freely available by LispWorks -- an invaluable reference.

See <https://www.lispworks.com/documentation/HyperSpec/Front/index.htm>.

2.4.3 Generating Executables

SBCL can generate stand-alone executables. The generated executables include the SBCL runtime itself, so no restrictions are placed on program functionality. For example, a deployed program can call `compile` and `load`, which requires the compiler to be present in the executable. For further information, `sb-ext:save-lisp-and-die`.

2.5 More SBCL Information

2.5.1 SBCL Homepage

The SBCL website at <http://www.sbcl.org/> has some general information, plus links to mailing lists devoted to SBCL, and to archives of these mailing lists. Subscribing to the mailing lists `sbcl-help` and `sbcl-announce` is recommended: both are fairly low-volume, and help you keep abreast with SBCL development.

2.5.2 Online Documentation

Documentation for non-ANSI extensions for various commands is available online from the SBCL executable itself. The extensions for functions which have their own command prompts (e.g. the debugger, and `inspect`) are documented in text available by typing `help` at their command prompts. The extensions for functions which don't have their own command prompt (such as `trace(0 1)`) are described in their documentation strings, unless your SBCL was compiled with an option not to include documentation strings, in which case the documentation strings are only readable in the source code.

2.5.3 Additional Documentation Files

Besides this user manual both SBCL source and binary distributions include some other SBCL-specific documentation files, which should be installed along with this manual on your system, e.g. in `/usr/local/share/doc/sbcl/`.

- `copying`: Licence and copyright summary.
- `credits`: Authorship information on various parts of SBCL.
- `install`: Covers installing SBCL from both source and binary distributions on your system, and also has some installation related troubleshooting information.
- `news`: Summarizes changes between various SBCL versions.

2.5.4 Internals Documentation

If you're interested in the development of the SBCL system itself, then subscribing to `sbcl-devel` is a good idea.

SBCL internals documentation -- besides comments in the source -- is available in the Web Archive:

<https://web.archive.org/web/20120814000933/http://sbcl-internals.cliki.net/index>.

Some low-level information describing the programming details of the conversion from CMUCL to SBCL is available in the `doc/FOR-CMUCL-DEVELOPERS` file.

2.6 More Common Lisp Information

2.6.1 Internet Community

IRC channels on <https://libera.chat/>:

- `#common-lisp`: "Common Lisp, the #1=(programmable . #1#) programming language"
- `#lispcafe`: "The Lisp Café; sit down, have a drink, chat about anything, and enjoy your stay. | <https://www.cliki.net/lispcafe> | Be insuperable to each other".
- `#sbcl`: "Steel Bank Common Lisp Dev Hangout"

You can use <https://web.libera.chat> or a normal IRC client.

Also, see https://www.reddit.com/r/Common_Lisp/, as well as <https://www.lisp.org> and <https://cliki.net>, which contain numerous pointers places in the net where lispers talk shop.

2.6.2 Third-party Libraries

For a wealth of information about free Common Lisp libraries and tools we recommend checking out *CLiki*: <https://cliki.net/>.

The most popular library manager is Quicklisp: <https://www.quicklisp.org/beta/>.

2.6.3 Common Lisp Books

If you're not a programmer and you're trying to learn, many introductory Lisp books are available. However, we don't have any standout favorites.

If you are an experienced programmer in other languages but need to learn about Common Lisp, some books stand out:

- Practical Common Lisp, by Peter Seibel
An excellent introduction to the language, covering both the basics and "advanced topics" like macros, CLOS, and packages. Available both in print format and on the web: <https://gigamonkeys.com/book/>.
- Paradigms Of Artificial Intelligence Programming, by Peter Norvig
Good information on general Common Lisp programming, and many nontrivial examples. Whether or not your work is AI, it's a very good book to look at.
- On Lisp, by Paul Graham
An in-depth treatment of macros, but not recommended as a first Common Lisp book, since it is slightly pre-ANSI so you need to be on your guard against non-standard usages, and since it doesn't really even try to cover the language as a whole, focusing solely on macros. Downloadable from <https://www.paulgraham.com/onlisp.html>.

- Object-Oriented Programming In Common Lisp, by Sonya Keene

With the exception of *Practical Common Lisp*, most introductory books don't emphasize CLOS. This one does. Even if you're very knowledgeable about object oriented programming in the abstract, it's worth looking at this book if you want to do any OO in Common Lisp. Some abstractions in CLOS (especially multiple dispatch) go beyond anything you'll see in most OO systems, and there are a number of lesser differences as well. This book tends to help with the culture shock.

- Art Of Metaobject Programming, by Gregor Kiczales et al.

Currently the prime source of information on the Common Lisp Metaobject Protocol, which is supported by SBCL. Section 2 (Chapters 5 and 6) are freely available at <http://mop.lisp.se/www.alu.org/mop/>.

2.7 History and Implementation of SBCL

You can work productively with SBCL without knowing or understanding anything about where it came from, how it is implemented, or how it extends the ANSI Common Lisp standard. However, a little knowledge can be helpful in order to understand error messages, to troubleshoot problems, to understand why some parts of the system are better debugged than others, and to anticipate which known bugs, known performance problems, and missing extensions are likely to be fixed, tuned, or added.

SBCL is descended from CMUCL, which is itself descended from Spice Lisp, including early implementations for the Mach operating system on the IBM RT, back in the 1980s. Some design decisions from that time are still reflected in the current implementation:

- The system expects to be loaded into a fixed-at-compile-time location in virtual memory, and also expects the location of all of its heap storage to be specified at compile time.
- The system overcommits memory, allocating large amounts of address space from the system (often more than the amount of virtual memory available) and then failing if it ends up using too much of the allocated storage.
- The system is implemented as a C program which is responsible for supplying low-level services and loading a Lisp `.core` file.

SBCL also inherited some newer architectural features from CMUCL. The most important is that on some architectures it has a generational garbage collector (GC), which has various implications (mostly good) for performance. These are discussed in another chapter, [Efficiency](#).

SBCL has diverged from CMUCL in that SBCL is now essentially a compiler-only implementation of Common Lisp. This is a change in implementation strategy, taking advantage of the freedom "any of these facilities might share the same execution strategy" guaranteed in [clhs 3.1](#) (Evaluation). It does not mean SBCL can't be used interactively, and in fact the change is largely invisible to the casual user, since SBCL still can and does execute code interactively by compiling it on the fly. (It is visible if you know how to look, like using `compiled-function-p`; and it is visible in the way that SBCL doesn't have many bugs which behave differently in interpreted code than in compiled code.) What it means is that in SBCL, the `eval` function only truly "interprets" a few

easy kinds of forms, such as symbols which are **boundp**. More complicated forms are evaluated by calling **compile** and then calling **funcall** on the returned result.

The direct ancestor of SBCL is the x86 port of CMUCL. This port was in some ways the most cobbled-together of all the CMUCL ports, since a number of strange changes had to be made to support the register-poor x86 architecture. Some things (like tracing and debugging) do not work particularly well there. SBCL should be able to improve in these areas (and has already improved in some other areas), but it takes a while.

On the x86 SBCL -- like the x86 port of CMUCL -- uses a *conservative* GC. This means that it doesn't maintain a strict separation between tagged and untagged data, instead treating some untagged data (e.g. raw floating point numbers) as possibly-tagged data and so not collecting any Lisp objects that they point to. This has some negative consequences for average time efficiency (though possibly no worse than the negative consequences of trying to implement an exact GC on a processor architecture as register-poor as the X86) and also has potentially unlimited consequences for worst-case memory efficiency. In practice, conservative garbage collectors work reasonably well, not getting anywhere near the worst case. But they can occasionally cause odd patterns of memory usage.

The fork from CMUCL was based on a major rewrite of the system bootstrap process. CMUCL has for many years tolerated a very unusual "build" procedure which doesn't actually build the complete system from scratch, but instead progressively overwrites parts of a running system with new versions. This quasi-build procedure can cause various bizarre bootstrapping hangups, especially when a major change is made to the system. It also makes the connection between the current source code and the current executable more tenuous than in other software systems -- it's easy to accidentally build a CMUCL system containing characteristics not reflected in the current version of the source code.

Other major changes since the fork from CMUCL include:

- SBCL has removed many CMUCL extensions, (e.g. IP networking, remote procedure call, Unix system interface, and X11 interface) from the core system. Most of these are available as contributed modules (distributed with SBCL) or third-party modules instead.
- SBCL has deleted or deprecated some nonstandard features and code complexity which helped efficiency at the price of maintainability. For example, the SBCL compiler no longer implements memory pooling internally (and so is simpler and more maintainable, but generates more garbage and runs more slowly).

3 Starting and Stopping

3.1 Starting SBCL

3.1.1 Running from Shell

To run SBCL, type `sbcℓ` at the command line.

You should end up in the toplevel *REPL* (read-eval-print loop), where you can interact with SBCL by typing expressions.

```

$ sbcl
This is SBCL 0.8.13.60, an implementation of ANSI Common Lisp.
More information about SBCL is available at <http://www.sbcl.org/>.

SBCL is free software, provided as is, with absolutely no warranty.
It is mostly in the public domain; some portions are provided under
BSD-style licenses. See the CREDITS and COPYING files in the
distribution for more information.
* (+ 2 2)
4
* (exit)
$

```

Also see [Command Line Options](#) and [Stopping SBCL](#).

3.1.2 Running from Emacs

To run SBCL as an `inferior-lisp` from Emacs, in your `.emacs` do something like:

```

;;; The SBCL binary and command-line arguments
(setq inferior-lisp-program "/usr/local/bin/sbcl --noinform")

```

For more information on using SBCL with Emacs, see [Editor Integration](#).

3.1.3 Shebang Scripts

Standard Unix tools that are interpreters follow a common command line protocol that is necessary to work with "shebang scripts". SBCL supports this via the `--script` command line option (see [Command Line Options](#)).

Example file (`hello.lisp`):

```

#!/usr/local/bin/sbcl --script
(write-line "Hello, World!")

```

Usage from the command line:

```

$ ./hello.lisp
Hello, World!

```

Note that SBCL skips the shebang line when it reads the file:

```

$ sbcl --script hello.lisp
Hello, World!

```

3.2 Stopping SBCL

3.2.1 Exit

SBCL can be stopped at any time by calling `sb-ext:exit`, optionally returning a specified numeric value to the calling process. See 2 for information about terminating individual threads.

- **[function] `sb-ext:exit`** *&key code abort (timeout *exit-timeout*)*

Terminates the process, causing SBCL to exit with `code`. `code` defaults to 0 when `abort` is false, and 1 when it is true.

When `abort` is false (the default), current thread is first unwound, **exit-hooks** are run, other threads are terminated, and standard output streams are flushed before SBCL calls `exit(3)` -- at which point `atexit(3)` functions will run. If multiple threads call `exit` with `abort` being false, the first one to call it will complete the protocol.

When `abort` is true, SBCL exits immediately by calling `_exit(2)` without unwinding stack, or calling exit hooks. Note that `_exit(2)` does not call `atexit(3)` functions unlike `exit(3)`.

Recursive calls to `exit` cause `exit` to behave as if `abort` was true.

`timeout` controls waiting for other threads to terminate when `abort` is nil. Once current thread has been unwound and **exit-hooks** have been run, spawning new threads is prevented and all other threads are terminated by calling `sb-thread:terminate-thread` on them. The system then waits for them to finish using `sb-thread:join-thread`, waiting at most a total `timeout` seconds for all threads to join. Those threads that do not finish in time are simply ignored while the exit protocol continues. `timeout` defaults to **exit-timeout**, which in turn defaults to 60. `timeout nil` means to wait indefinitely.

Note that `timeout` applies only to `sb-thread:join-thread`, not **exit-hooks**. Since `sb-thread:terminate-thread` is asynchronous, getting multithreaded application termination with complex cleanups right using it can be tricky. To perform an orderly synchronous shutdown use an exit hook instead of relying on implicit thread termination.

Consequences are unspecified if serious conditions occur during `exit` excepting errors from **exit-hooks**, which cause warnings and stop execution of the hook that signaled, but otherwise allow the exit process to continue normally.

3.2.2 End of File

By default SBCL also exits on end of input, caused either by user pressing `Control-D` on an attached terminal, or end of input when using SBCL as part of a shell pipeline.

3.2.3 Saving a Core Image

SBCL has the ability to save its state as a file for later execution. This functionality is important for its bootstrapping process, and is also provided as an extension to the user.

- **[function] `sb-ext:save-lisp-and-die`** *core-file-name &key (toplevel #'toplevel-init toplevel-supplied) (executable nil) (save-runtime-options nil) (callable-exports nil) (purify t) (root-structures nil) (environment-name "auxiliary") (compression nil)*

Save a "core image", i.e. enough information to restart a Lisp process later in the same state, in the file of the specified name. Only global state is preserved: the stack is unwound in the process.

The following *&key* arguments are defined:

- `:toplevel`

The function to run when the created core file is resumed. The default function handles command line `toplevel` option processing and runs the top level read-eval-print loop. This function returning is equivalent to (`sb-ext:exit :code 0`) being called.

`toplevel` functions should always provide an `abort` restart: otherwise code they call will run without one.

- `:executable`

If true, arrange to combine the SBCL runtime and the core image to create a standalone executable. If false (the default), the core image will not be executable on its own. Executable images always behave as if they were passed the `--noinform` runtime option. If `:executable` is `:elf-object`, then the resulting core will be wrapped in a `.o` which requires further linking. (EXPERIMENTAL)

- `:save-runtime-options`

If true, values of runtime options `--dynamic-space-size` and `--control-stack-size` that were used to start SBCL are stored in the standalone executable, and restored when the executable is run. This also inhibits normal runtime option processing, causing all command line arguments to be passed to the `toplevel`. If `:accept-runtime-options` then `--dynamic-space-size` and `--control-stack-size` are still processed by the runtime. Meaningless if `:executable` is `nil`.

- `:callable-exports`

This should be a list of symbols to be initialized to the appropriate alien callables on startup. All exported symbols should be present as global symbols in the symbol table of the runtime before the saved core is loaded. When this list is non-empty, the `:toplevel` argument cannot be supplied.

- `:purify`

If true (the default), then some objects in the restarted core will be memory-mapped as read-only. Among those objects are numeric vectors that were determined to be compile-time constants, and any immutable values according to the language specification such as symbol names.

- `:root-structures`

This should be a list of the main entry points in any newly loaded systems. This need not be supplied, but locality and/or gc performance may be better if they are. This has two different but related meanings: If `:purify` is true - and only for `cheneygc` - the root structures are those which anchor the set of objects moved into static space. On `gencgc` - and only on platforms supporting immobile code - these are the functions and/or function-names which commence a depth-first scan of code when reordering based on the statically observable call chain. The complete set of reachable objects is not affected per se. This argument is meaningless if neither enabling precondition holds.

- `:environment-name`

This has no purpose; it is accepted only for legacy compatibility.

- `:compression`

This is only meaningful if the runtime was built with the `:sb-core-compression` feature enabled. If `nil` (the default), saves to uncompressed core files. If `:sb-core-compression` was enabled at build-time, the argument may also be an integer from -7 to 22, corresponding to `zstd` compression levels, or `t` (which is equivalent to the default compression level, 9).

- `:application-type`

Present only on Windows and is meaningful only with `:executable t`. Specifies the subsystem of the executable, `:console` or `:gui`. The notable difference is that `:gui` doesn't automatically create a console window. The default is `:console`.

The save/load process changes the values of some global variables:

- `*standard-output*`, `*debug-io*`, etc

Everything related to open streams is necessarily changed, since the OS won't let us preserve a stream across save and load.

- `*default-pathname-defaults*`

This is reinitialized to reflect the working directory where the saved core is loaded.

`save-lisp-and-die` interacts with `sb-alien:load-shared-object`: see its documentation for details.

On threaded platforms only a single thread may remain running after `sb-ext:*save-hooks*` have run. Applications using multiple threads can be `save-lisp-and-die` friendly by registering a `save-hook` that quits any additional threads, and an `init-hook` that restarts them.

This implementation is not as polished and painless as you might like: * It corrupts the current Lisp image enough that the current process needs to be killed afterwards. This can be worked around by forking another process that saves the core. * There is absolutely no binary compatibility of core images between different runtime support programs. Even runtimes built from the same sources at different times are treated as incompatible for this purpose. This isn't because we like it this way, but just because there don't seem to be good quick fixes for either limitation and no one has been sufficiently motivated to do lengthy fixes.

- **[variable]** `sb-ext:*save-hooks*` (*swank/backend:deinit-log-output*)

A list of function designators which are called in an unspecified order before creating a saved core image.

Unused by SBCL itself: reserved for user and applications.

In cases where the standard initialization files have already been loaded into the saved core, and alternative ones should be used (or none at all), SBCL allows customizing the initfile pathname computation.

- **[variable]** `sb-ext:*sysinit-pathname-function*` *#<function sb-impl::sysinit-pathname>*

Designator for a function of zero arguments called to obtain a pathname designator for the default sysinit file, or `nil`. If the function returns `nil`, no sysinit file is used unless one has been specified on the command-line.

- **[variable]** `sb-ext:*userinit-pathname-function*` *#<function sb-impl::userinit-pathname>*

Designator for a function of zero arguments called to obtain a pathname designator or a stream for the default userinit file, or `nil`. If the function returns `nil`, no userinit file is used unless one has been specified on the command-line.

To facilitate distribution of SBCL applications using external resources, the filesystem location of the SBCL core file being used is available from Lisp.

- **[variable]** `sb-ext:*core-pathname*` *"<site-specific>"*

The absolute pathname of the running SBCL core.

3.2.4 Exit on Errors

SBCL can also be configured to exit if an unhandled error occurs, which is mainly useful for acting as part of a shell pipeline; doing so under most other circumstances would mean giving up large parts of the flexibility and robustness of Common Lisp. See [Debugger Entry](#) and the command line option `--disable-debugger` in [Runtime Options](#).

3.3 Command Line Options

Command line options can be considered an advanced topic; for ordinary interactive use, no command line arguments should be necessary.

In order to understand the command line argument syntax for SBCL, it is helpful to understand that the SBCL system is implemented as two components, a low-level runtime environment written in C and a higher-level system written in Common Lisp itself. Some command line arguments are processed during the initialization of the low-level runtime environment, some command line arguments are processed during the initialization of the Common Lisp system, and any remaining command line arguments are passed on to user code.

The full, unambiguous syntax for invoking SBCL at the command line is:

```
sbcl <runtime-option>* --end-runtime-options \  
    <toplevel-option>* --end-toplevel-options \  
    <user-option>*
```

For convenience, `--end-runtime-options` and `--end-toplevel-options` can be omitted, which can be convenient when you are running the program interactively, and you can see that no ambiguities are possible with the option values you are using. Omitting these elements

is probably a bad idea for any batch file where any of the options are under user control, since it makes it impossible for SBCL to detect erroneous command line input, so that erroneous command line arguments will be passed on to the user program even if they were intended for the runtime system or the Lisp system.

3.3.1 Runtime Options

- `--core <corefilename>`

Run the specified Lisp core file instead of the default. Note that if the Lisp core file is a user-created core file, it may run a nonstandard toplevel which does not recognize the standard toplevel options.

- `--dynamic-space-size <megabytes>`

Size of the dynamic space reserved on startup in megabytes. Default value is platform dependent.

- `--control-stack-size <megabytes>`

Size of control stack reserved for each thread in megabytes. Default value is 2.

- `--tls-limit <positive integer>`

Maximum number of thread-local symbols in threaded builds. Default value is 4096.

- `--noinform`

Suppress the printing of any banner or other informational message at startup. This makes it easier to write Lisp programs which work cleanly in Unix pipelines. See also the `--noprint` and `--disable-debugger` options.

- `--disable-ldb`

Disable the low-level debugger. Only effective if SBCL is compiled with `ldb`.

- `--lose-on-corruption`

There are some dangerous low-level errors (for instance, control stack exhausted, memory fault) that (or whose handlers) can corrupt the image. By default, SBCL prints a warning, then tries to continue and handle the error in Lisp, but this will not always work, and SBCL may malfunction or even hang. With this option, upon encountering such an error, SBCL will exit instead of invoking `ldb` (if present and enabled).

- `--script <filename>`

As a *runtime* option, this is equivalent to `--noinform --disable-ldb --lose-on-corruption --end-runtime-options --script <filename>`. See the description of `--script` as a *toplevel* option below. If there are no other command line arguments following `--script`, the filename argument can be omitted.

- `--merge-core-pages`

When platform support is present, provide hints to the operating system that identical pages may be shared between processes until they are written to. This can be useful to

reduce the memory usage on systems with multiple SBCL processes started from similar but differently-named core files, or from compressed cores. Without platform support, do nothing. By default only compressed cores trigger hinting.

- `--no-merge-core-pages`

Ensures that no sharing hint is provided to the operating system.

- `--help`

Print some basic information about SBCL, then exit.

- `--version`

Print SBCL's version information, then exit.

In the future, runtime options may be added to control behaviour such as lazy allocation of memory.

Runtime options, including any `--end-runtime-options` option, are stripped out of the command line before the Lisp toplevel logic gets a chance to see it.

3.3.2 Toplevel Options

- `--sysinit <filename>`

Load `filename` instead of the default system initialization file (see [Initialization Files](#)).

- `--no-sysinit`

Don't load a system-wide initialization file. If this option is given, the `--sysinit` option is ignored.

- `--userinit <filename>`

Load `filename` instead of the default user initialization file (see [Initialization Files](#).)

- `--no-userinit`

Don't load a user initialization file. If this option is given, the `--userinit` option is ignored.

- `--eval <command>`

After executing any initialization file, but before starting the read-eval-print loop on standard input, read and evaluate `command`. More than one `--eval` option can be used, and all will be read and executed, in the order they appear on the command line.

- `--load <filename>`

This is equivalent to `--eval '(load "<filename>")'`. The special syntax is intended to reduce quoting headaches when invoking SBCL from shell scripts.

- `--noprint`

When ordinarily the toplevel "read-eval-print loop" would be executed, execute a "read-eval loop" instead, i.e. don't print a prompt and don't echo results. Combined with the

`--noinform` runtime option, this makes it easier to write Lisp "scripts" which work cleanly in Unix pipelines.

- `--disable-debugger`

By default when SBCL encounters an error, it enters the builtin debugger, allowing interactive diagnosis and possible intercession. This option disables the debugger, causing errors to print a backtrace and exit with status 1 instead. When given, this option takes effect before loading of initialization files or processing `--eval` and `--load` options. See [sb-ext:disable-debugger](#) and [Debugger Entry](#).

- `--script <filename>`

Implies `--no-userinit` `--no-sysinit` `--disable-debugger` `--end-toplevel-options`.

Causes the system to load the specified file instead of entering the read-eval-print-loop, and exit afterwards. If the file begins with a shebang line, it is ignored.

If there are no other command line arguments following, the filename can be omitted: this causes the script to be loaded from standard input instead. Shebang lines in standard input script are currently *not* ignored.

In either case, if there is an unhandled error (e.g. end of file, or a broken pipe) on either standard input, standard output, or standard error, the script silently exits with code

0. This allows e.g. safely piping output from SBCL to `head -n1` or similar.

Additionally, the option sets `*compile-verbose*` and `*load-verbose*` to `nil` while loading the file to avoid potentially verbose diagnostic messages printed on the standard output.

3.4 Initialization Files

SBCL processes initialization files with `read` and `eval`, not `load`; hence initialization files can be used to set startup `*package*` and `*readtable*`, and for proclaiming a global optimization policy.

- **System Initialization File:** Defaults to `$SBCL_HOME/sbclrc`, or if that doesn't exist to `/etc/sbclrc`. Can be overridden with the command line option `--sysinit` or `--no-sysinit` (see [Toplevel Options](#)).

The system initialization file is intended for system administrators and software packagers to configure locations of installed third party modules, etc.

- **User Initialization File:** Defaults to `$HOME/.sbclrc`. Can be overridden with the command line option `--userinit` or `--no-userinit` (see [Toplevel Options](#)).

The user initialization file is intended for personal customizations, such as loading certain modules at startup, defining convenience functions to use in the REPL, handling automatic recompilation of FASLs (see [FASL format](#)), etc.

Neither initialization file is required.

3.5 Initialization and Exit Hooks

SBCL provides hooks into the system initialization and exit.

- **[variable]** `sb-ext:*init-hooks*` *nil*

A list of function designators which are called in an unspecified order when a saved core image starts up, after the system itself has been initialized, but before non-user threads such as the finalizer thread have been started.

Unused by SBCL itself: reserved for user and applications.

- **[variable]** `sb-ext:*exit-hooks*` *nil*

A list of function designators which are called in an unspecified order when SBCL process exits.

Unused by SBCL itself: reserved for user and applications.

Using `(sb-ext:exit :abort t)`, or calling `exit(3)` directly circumvents these hooks.

4 Compiler

This chapter will discuss most compiler issues other than efficiency, including compiler error messages, the SBCL compiler's unusual approach to type safety in the presence of type declarations, the effects of various compiler optimization policies, and the way that inlining and open coding may cause optimized code to differ from a naive translation. Efficiency issues are sufficiently varied and separate that they have their own chapter, [Efficiency](#).

4.1 Diagnostic Messages

4.1.1 Controlling Verbosity

The compiler can be quite verbose in its diagnostic reporting, rather more than some users would prefer -- the amount of noise emitted can be controlled, however.

To control emission of compiler diagnostics (of any severity other than `error(0 1)`: [Diagnostic Severity](#)) use the `sb-ext:muffle-conditions` and `sb-ext:unmuffle-conditions` declarations, specifying the type of condition that is to be muffled (the muffling is done using an associated `muffle-warning` restart).

Global control:

```
;;; Muffle compiler-notes globally
(declaim (sb-ext:muffle-conditions sb-ext:compiler-note))
```

Local control:

```
;;; Muffle compiler-notes based on lexical scope
(defun foo (x)
  (declare (optimize speed) (fixnum x)
           (sb-ext:muffle-conditions sb-ext:compiler-note))
  (values (* x 5) ; no compiler note from this
```

```
(locally
  (declare (sb-ext:unmuffle-conditions sb-ext:compiler-note))
  ;; this one gives a compiler note
  (* x -5)))
```

- **[declaration]** `sb-ext:muffle-conditions` &REST TYPES

Muffle the diagnostic messages that would be caused by compile-time signals of given types.

- **[declaration]** `sb-ext:unmuffle-conditions` &REST TYPES

Cancel the effect of a previous `sb-ext:muffle-conditions` declaration.

Various details of *how* the compiler messages are printed can be controlled via the alist `sb-ext:*compiler-print-variable-alist*`.

- **[variable]** `sb-ext:*compiler-print-variable-alist*` *nil*

An association list describing new bindings for special variables to be used by the compiler for error-reporting, etc. E.g. `((*print-length* . 10) (*print-level* . 6) (*print-pretty* . nil))`.

The variables in the *car* positions are bound to the values in the *cdr* during the execution of some debug commands. When evaluating arbitrary expressions in the debugger, the normal values of the printer control variables are in effect.

Initially empty, `*compiler-print-variable-alist*` is typically used to specify bindings for printer control variables.

For information about muffling warnings signaled outside of the compiler, see @CUSTOMIZATION-HOOKS-FOR-USERS.

4.1.2 Diagnostic Severity

There are four levels of compiler diagnostic severity:

- error
- warning
- style warning
- note

The first three levels correspond to condition classes which are defined in the ANSI standard for Common Lisp and which have special significance to the `compile` and `compile-file` functions. These levels of compiler error severity occur when the compiler handles conditions of these classes.

The fourth level of compiler error severity, *note*, corresponds to the `sb-ext:compiler-note`, and is used for problems which are too mild for the standard condition classes, typically hints

about how efficiency might be improved. The `sb-ext:code-deletion-note`, a subtype of `sb-ext:compiler-note`, is signalled when the compiler deletes user-supplied code after proving that the code in question is unreachable.

Future work for SBCL includes expanding this hierarchy of types to allow more fine-grained control over emission of diagnostic messages.

- **[condition]** `sb-ext:compiler-note`

Root of the hierarchy of conditions representing information discovered by the compiler that the user might wish to know, but which does not merit a `style-warning` (or any more serious condition).

- **[condition]** `sb-ext:code-deletion-note` *sb-int:simple-compiler-note*

A condition type signalled when the compiler deletes code that the user has written, having proved that it is unreachable.

4.1.3 Understanding Compiler Diagnostics

The messages emitted by the compiler contain a lot of detail in a terse format, so they may be confusing at first. The messages will be illustrated using this example program:

```
(defmacro zoq (x)
  `(roq (ploq (+ ,x 3))))

(defun foo (y)
  (declare (symbol y))
  (zoq y))
```

The main problem with this program is that it is trying to add 3 to a symbol. Note also that the functions `roq` and `ploq` aren't defined anywhere.

Parts of a Compiler Diagnostic When processing this program, the compiler will produce this warning:

```
; file: /tmp/foo.lisp
; in: DEFUN FOO
;   (ZOQ Y)
; --> ROQ PLOQ
; ==>
;   (+ Y 3)
;
; caught WARNING:
;   Asserted type NUMBER conflicts with derived type (VALUES SYMBOL &OPTIONAL).
```

In this example we see each of the six possible parts of a compiler diagnostic:

- `file: /tmp/foo.lisp` is the name of the file that the compiler read the relevant code from. The file name is displayed because it may not be immediately obvious when there is an error during compilation of a large system, especially when `with-compilation-unit(0 1)` is used to delay undefined warnings.

- `in: DEFUN F00` is the definition top level form responsible for the diagnostic. It is obtained by taking the first two elements of the enclosing form whose first element is a symbol beginning with `def`. If there is no such enclosing `def` form, then the outermost form is used. If there are multiple `def` forms, then they are all printed from the outside in, separated by `=>s`. In this example, the problem was in the `defun` for `foo`.
- `(zoq y)` is the *original source* form responsible for the diagnostic. Original source means that the form directly appeared in the original input to the compiler, i.e. in the lambda passed to `compile` or in the top level form read from the source file. In this example, the expansion of the `zoq` macro was responsible for the message.
- `--> roq ploq` This is the *processing path* that the compiler used to produce the code that caused the message to be emitted. The processing path is a representation of the evaluated forms enclosing the actual source that the compiler encountered when processing the original source. The path is the first element of each form, or the form itself if the form is not a list. These forms result from the expansion of macros or source-to-source transformation done by the compiler. In this example, the enclosing evaluated forms are the calls to `roq` and `ploq`. These calls resulted from the expansion of the `zoq` macro.
- `==> (+ y 3)` is the *actual source* responsible for the diagnostic. If the actual source appears in the explanation, then we print the next enclosing evaluated form, instead of printing the actual source twice. (This is the form that would otherwise have been the last form of the processing path.) In this example, the problem is with the evaluation of the reference to the variable `y`.
- `caught WARNING: Asserted type NUMBER conflicts with derived type (VALUES SYMBOL &OPTIONAL).` is the *explanation* of the problem. In this example, the problem is that, while the call to `+(0 1)` requires that its arguments are all of type `number`, the compiler has derived that `Y` will evaluate to a `symbol`. Note that `(values symbol &optional)` expresses that `y` evaluates to precisely one value.

Note that each part of the message is distinctively marked:

- `file:` and `in:` mark the file and definition, respectively.
- The original source is an indented form with no prefix.
- Each line of the processing path is prefixed with `-->`.
- The actual source form is indented like the original source, but is marked by a preceding `==>` line. (FIXME: no it isn't.)
- The explanation is prefixed with the diagnostic severity, which can be `caught ERROR:`, `caught WARNING:`, `caught STYLE-WARNING:`, or `note:`.

Each part of the message is more specific than the preceding one. If consecutive messages are for nearby locations, then the front part of the messages would be the same. In this case, the compiler omits as much of the second message as in common with the first. For example:

```
; file: /tmp/foo.lisp
; in: DEFUN F00
;   (ZOQ Y)
; --> ROQ
```

```

; ==>
; (PLOQ (+ Y 3))
;
; caught STYLE-WARNING:
; undefined function: PLOQ

; ==>
; (ROQ (PLOQ (+ Y 3)))
;
; caught STYLE-WARNING:
; undefined function: ROQ

```

In this example, the file, definition and original source are identical for the two messages, so the compiler omits them in the second message. If consecutive messages are entirely identical, then the compiler prints only the first message, followed by: [Last message occurs <repeats> times] where <repeats> is the number of times the message was given.

If the source was not from a file, then no file line is printed. If the actual source is the same as the original source, then the processing path and actual source will be omitted. If no forms intervene between the original source and the actual source, then the processing path will also be omitted.

Original and Actual Source The *original source* displayed will almost always be a list. If the actual source for an message is a symbol, the original source will be the immediately enclosing evaluated list form. So even if the offending symbol does appear in the original source, the compiler will print the enclosing list and then print the symbol as the actual source (as though the symbol were introduced by a macro.)

When the *actual source* is displayed (and is not a symbol), it will always be code that resulted from the expansion of a macro or a source-to-source compiler optimization. This is code that did not appear in the original source program; it was introduced by the compiler.

Keep in mind that when the compiler displays a source form in an diagnostic message, it always displays the most specific (innermost) responsible form. For example, compiling this function

```
(defun bar (x) (let (a) (declare (fixnum a)) (setq a (foo x)) a))
```

gives this error message

```

; file: /tmp/foo.lisp
; in: DEFUN BAR
; (LET (A)
; (DECLARE (FIXNUM A))
; (SETQ A (FOO X))
; A)
;
; caught WARNING:
; Asserted type FIXNUM conflicts with derived type (VALUES NULL &OPTIONAL).

```

This message is not saying that there is a problem somewhere in this `let` -- it is saying that there is a problem with the `let` itself. In this example, the problem is that `a`'s `nil` initial value is not a `fixnum`.

Processing Path The processing path is mainly useful for debugging macros, so if you don't write macros, you can probably ignore it. Consider this example:

```
(defun foo (n)
  (dotimes (i n *undefined*)))
```

Compiling results in this error message:

```
; in: DEFUN F00
;   (DOTIMES (I N *UNDEFINED*))
; --> DO BLOCK LET TAGBODY RETURN-FROM
; ==>
;   (PROGN *UNDEFINED*)
;
; caught WARNING:
;   undefined variable: *UNDEFINED*
```

Note that **do** appears in the processing path. This is because **dotimes** expands into:

```
(do ((i 0 (1+ i)) (#:g1 n))
    ((>= i #:g1) *undefined*)
    (declare (type unsigned-byte i)))
```

The rest of the processing path results from the expansion of **do**:

```
(block nil
  (let ((i 0) (#:g1 n))
    (declare (type unsigned-byte i))
    (tagbody (go #:g3)
      #:g2 (psetq i (1+ i))
      #:g3 (unless (>= i #:g1) (go #:g2))
      (return-from nil (progn *undefined*)))))
```

In this example, the compiler descended into the **block**, **let**, **tagbody** and **return-from** to reach the **progn** printed as the actual source. This is a place where the "actual source appears in explanation" rule was applied. The innermost actual source form was the symbol *undefined* itself, but that also appeared in the explanation, so the compiler backed out one level.

4.2 Handling of Types

One of the most important features of the SBCL compiler (similar to the original CMUCL compiler, also known as *Python*) is its fairly sophisticated understanding of the Common Lisp type system and its conservative approach to the implementation of type declarations.

These two features reward the use of type declarations throughout development, even when high performance is not a concern. Also, as discussed in the chapter on performance (see [Efficiency](#)), the use of appropriate type declarations can be very important for performance as well.

The SBCL compiler also has a greater knowledge of the Common Lisp type system than other compilers. Support is incomplete only for types involving the **satisfies** type specifier.

4.2.1 Declarations as Assertions

The SBCL compiler treats type declarations differently from most other Lisp compilers. Under default compilation policy the compiler doesn't blindly believe type declarations, but considers them assertions about the program that should be checked: all type declarations that have not been proven to always hold are asserted at runtime.

Remaining bugs in the compiler's handling of types unfortunately provide some exceptions to this rule, see [Implementation Limitations](#).

CLOS slot types form a notable exception. Types declared using the `:type` slot option in `defclass` are asserted if and only if the class was defined in *safe code* and the slot access location is in *safe code* as well. This laxness does not pose any internal consistency issues, as the CLOS slot types are not available for the type inferencer, nor do CLOS slot types provide any efficiency benefits.

There are three type checking policies available in SBCL, selectable via `optimize` declarations.

- **Full Type Checks**

All declarations are considered assertions to be checked at runtime, and all type checks are precise. The default compilation policy provides full type checks.

Used when `(or (>= safety 2) (>= safety speed 1))`.

- **Weak Type Checks**

Declared types may be simplified into faster to check supertypes: for example, `(or (integer -17 -7) (integer 7 17))` is simplified into `(integer -17 17)`.

Note: It is relatively easy to corrupt the heap when weak type checks are used if the program contains type-errors.

Used when `(and (< safety 2) (< safety speed))`.

- **No Type Checks**

All declarations are believed without assertions. Also disables argument count and array bounds checking.

Note: any type errors in code where type checks are not performed are liable to corrupt the heap.

Used when `(= safety 0)`.

4.2.2 Precise Type Checking

Precise checking means that the check is done as though `typep` had been called with the exact type specifier that appeared in the declaration.

If a variable is declared to be `(integer 3 17)`, then its value must always be an integer between 3 and 17. If multiple type declarations apply to a single variable, then all the declarations must be correct; it is as though all the types were intersected producing a single `and` type specifier.

To gain maximum benefit from the compiler's type checking, you should always declare the types of function arguments and structure slots as precisely as possible. This often involves the

use of `or(0 1)`, `member(0 1)`, and other list-style type specifiers.

4.2.3 Getting Existing Programs to Run

Since SBCL's compiler does much more comprehensive type checking than most Lisp compilers, SBCL may detect type errors in programs that have been debugged using other compilers. These errors are mostly incorrect declarations, although compile-time type errors can find actual bugs if parts of the program have never been tested.

Some incorrect declarations can only be detected by run-time type checking. It is very important to initially compile a program with full type checks (high **safety** optimization) and then test this safe version. After the checking version has been tested, then you can consider weakening or eliminating type checks. *This applies even to previously debugged programs* because the SBCL compiler does much more type inference than other Common Lisp compilers, so an incorrect declaration can do more damage.

The most common problem is with variables whose constant initial value doesn't match the type declaration. Incorrect constant initial values will always be flagged by a compile-time type error, and they are simple to fix once located. Consider this code fragment:

```
(prog (foo)
  (declare (fixnum foo))
  (setq foo ...)
  ...)
```

Here `foo` is given an initial value of `nil` but is declared to be a **fixnum**. Even if it is never read, the initial value of a variable must match the declared type. There are two ways to fix this problem. Change the declaration

```
(prog (foo)
  (declare (type (or fixnum null) foo))
  (setq foo ...)
  ...)
```

or change the initial value

```
(prog ((foo 0))
  (declare (fixnum foo))
  (setq foo ...)
  ...)
```

It is generally preferable to change to a legal initial value rather than to weaken the declaration, but sometimes it is simpler to weaken the declaration than to try to make an initial value of the appropriate type.

Another declaration problem occasionally encountered is incorrect declarations on **defmacro** arguments. This can happen when a function is converted into a macro. Consider this macro:

```
(defmacro my-1+ (x)
  (declare (fixnum x))
  `(the fixnum (1+ ,x)))
```

Although legal and well-defined Common Lisp code, this meaning of this definition is almost certainly not what the writer intended. For example, this call is illegal:

```
(my-1+ (+ 4 5))
```

This call is illegal because the argument to the macro is `(+ 4 5)`, which is a `list(0 1)`, not a `fixnum`. Because of macro semantics, it is hardly ever useful to declare the types of macro arguments. If you really want to assert something about the type of the result of evaluating a macro argument, then put a `the` in the expansion:

```
(defmacro my-1+ (x)
  `(the fixnum (1+ (the fixnum ,x))))
```

In this case, it would be stylistically preferable to change this macro back to a function and declare it inline.

Some more subtle problems are caused by incorrect declarations that can't be detected at compile time. Consider this code:

```
(do ((pos 0 (position #a string :start (1+ pos))))
    ((null pos))
  (declare (fixnum pos))
  ...)
```

Although `pos` is almost always a `fixnum`, it is `nil` at the end of the loop. If this example is compiled with full type checks (the default), then running it will signal a type error at the end of the loop. If compiled without type checks, the program will go into an infinite loop (or perhaps `position` will complain because `(1+ nil)` isn't a sensible start.) Why? Because if you compile without type checks, the compiler just quietly believes the type declaration. Since the compiler believes that `pos` is always a `fixnum`, it believes that `pos` is never `nil`, so `(null pos)` is never true, and the loop exit test is optimized away. Such errors are sometimes flagged by unreachable code notes, but it is still important to initially compile and test any system with full type checks, even if the system works fine when compiled using other compilers.

In this case, the fix is to weaken the type declaration to `(or fixnum null)`. (Actually, this declaration is unnecessary in SBCL, since it already knows that `position` returns a non-negative `fixnum` or `nil`.)

Note that there is usually little performance penalty for weakening a declaration in this way. Any numeric operations in the body can still assume that the variable is a `fixnum`, since `nil` is not a legal numeric argument. Another possible fix would be to say:

```
(do ((pos 0 (position #a string :start (1+ pos))))
    ((null pos))
  (let ((pos pos))
    (declare (fixnum pos))
    ...))
```

This would be preferable in some circumstances, since it would allow a non-standard representation to be used for the local `pos` variable in the loop body.

4.2.4 Implementation Limitations

If an `ftype` is placed after the function definition the function won't perform any type checks, and the calls to the function will blindly trust the declared types. `(optimize (debug 3))` will not trust any `ftype` declarations.

4.3 Compiler Policy

Compiler policy is controlled by the `optimize` declaration, supporting all ANSI optimization qualities (`debug`, `safety`, `space`, and `speed`). (A deprecated extension `sb-ext:inhibit-warnings` is still supported but liable to go away at any time.)

For effects of various optimization qualities on type-safety and debuggability see [Declarations as Assertions](#) and [Debugger Policy Control](#).

Ordinarily, when the speed quality is high, the compiler emits notes to notify the programmer about its inability to apply various optimizations. For selective muffling of these notes, see [Controlling Verbosity](#).

The value of `space` mostly influences the compiler's decision whether to inline operations, which tend to increase the size of programs. Use the value `0` with caution, since it can cause the compiler to inline operations so indiscriminately that the net effect is to slow the program by causing cache misses or even swapping.

- **[function]** `sb-ext:describe-compiler-policy` *&optional spec*

Print all global optimization settings, augmented by *spec*.

- **[function]** `sb-ext:restrict-compiler-policy` *&optional quality (min 0) (max 3)*

Assign a minimum value to an optimization quality. *quality* is the name of the optimization quality to restrict, *min* (defaulting to zero) is the minimum allowed value, and *max* (defaults to 3) is the maximum.

Returns the alist describing the current policy restrictions.

If *quality* is `nil` or not given, nothing is done.

Otherwise, if *min* is zero or *max* is 3 or neither are given, any existing restrictions of *quality* are removed.

See also `:policy` option in `with-compilation-unit(0 1)`.

- **[macro]** `with-compilation-unit` *options &body body*

Affects compilations that take place within its dynamic extent. It is intended to be eg. wrapped around the compilation of all files in the same system.

Following options are defined:

- `:override` *<boolean-form>*

One of the effects of this form is to delay undefined warnings until the end of the form, instead of giving them at the end of each compilation. If *override* is `nil` (the default), then the outermost `with-compilation-unit` form grabs the undefined

warnings. Specifying `:override true` causes that form to grab any enclosed warnings, even if it is enclosed by another `with-compilation-unit`.

- `:policy <optimize-declaration-form>`

Provides dynamic scoping for global compiler optimization qualities and restrictions, limiting effects of subsequent `optimize` proclamations and calls to `sb-ext:restrict-compiler-policy` to the dynamic scope of body.

If `:override` is false, the specified `:policy` is merged with current global policy. If `:override` is true, current global policy, including any restrictions, is discarded in favor of the specified `:policy`.

Supplying `:policy nil` is equivalent to the option not being supplied at all, i.e. dynamic scoping of policy does not take place.

This option is an SBCL-specific experimental extension: Interface subject to change.

- `:source-namestring <namestring-form>`

Attaches the value returned by the `<namestring-form>` to the internal debug-source information as the namestring of the source file. Normally the namestring of the input-file for `compile-file` is used: this option can be used to provide source-file information for functions compiled using `compile`, or to override the input-file of `compile-file`.

If both an outer and an inner `with-compilation-unit` provide a `:source-namestring`, the inner one takes precedence. Unaffected by `:override`.

This is an SBCL-specific extension.

- `:source-plist <plist-form>`

Attaches the value returned by the `<plist-form>` to internal debug-source information of functions compiled in within the dynamic extent of body.

Primarily for use by development environments, in order to eg. associate function definitions with editor-buffers. Can be accessed using `sb-introspect:definition-source-plist`.

If an outer `with-compilation-unit` form also provide a `source-plist`, it is appended to the end of the provided `source-plist`. Unaffected by `:override`.

This is an SBCL-specific extension.

Examples:

```
;; Prevent proclamations from the file leaking, and restrict  
;; SAFETY to 3 -- otherwise uses the current global policy.  
(with-compilation-unit (:policy '(optimize))  
  (restrict-compiler-policy 'safety 3)  
  (load "foo.lisp"))
```

```
;; Using default policy instead of the current global one,  
;; except for DEBUG 3.
```

```
(with-compilation-unit (:policy '(optimize debug)
                        :override t)
  (load "foo.lisp"))

;; Same as if :POLICY had not been specified at all: SAFETY 3
;; proclamation leaks out from WITH-COMPILATION-UNIT.
(with-compilation-unit (:policy nil)
  (declaim (optimize safety))
  (load "foo.lisp"))
```

4.4 Compiler Errors

4.4.1 Type Errors at Compile Time

If the compiler can prove at compile time that some portion of the program cannot be executed without a type error, then it will give a warning at compile time.

It is possible that the offending code would never actually be executed at run-time due to some higher level consistency constraint unknown to the compiler, so a type warning doesn't always indicate an incorrect program.

For example, consider this code fragment:

```
(defun raz (foo)
  (let ((x (case foo
             (:this 13)
             (:that 9)
             (:the-other 42))))
    (declare (fixnum x))
    (foo x)))
```

Compilation produces this warning:

```
; in: DEFUN RAZ
;   (CASE FOO (:THIS 13) (:THAT 9) (:THE-OTHER 42))
; --> LET COND IF COND IF COND IF
; ==>
;   (COND)
;
; caught WARNING:
;   This is not a FIXNUM:
;   NIL
```

In this case, the warning means that if `foo` isn't any of `:this`, `:that` or `:the-other`, then `x` will be initialized to `nil`, which the `fixnum` declaration makes illegal. The warning will go away if `ecase` is used instead of `case`, or if `:THE-OTHER` is changed to `t`.

This sort of spurious type warning happens moderately often in the expansion of complex macros and in inline functions. In such cases, there may be dead code that is impossible to correctly execute. The compiler can't always prove this code is dead (could never be executed), so it compiles the erroneous code (which will always signal an error if it is executed) and gives a warning.

4.4.2 Errors During Macroexpansion

The compiler handles errors that happen during macroexpansion, turning them into compiler errors. If you want to debug the error (to debug a macro), you can set **break-on-signals** to `error(0 1)`. For example, this definition:

```
(defun foo (e l)
  (do ((current l (cdr current))
      ((atom current) nil))
      (when (eq (car current) e) (return current))))
```

gives this error:

```
; in: DEFUN F00
;   (DO ((CURRENT L (CDR CURRENT))
;       ((ATOM CURRENT) NIL))
;       (WHEN (EQ (CAR CURRENT) E) (RETURN CURRENT)))
;
; caught ERROR:
;   (in macroexpansion of (DO # #))
;   (hint: For more precise location, try *BREAK-ON-SIGNALS*.)
;   DO step variable is not a symbol: (ATOM CURRENT)
```

4.4.3 Read Errors

SBCL's compiler does not attempt to recover from read errors when reading a source file, but instead just reports the offending character position and gives up on the entire source file.

4.5 Open Coding and Inline Expansion

Since Common Lisp forbids the redefinition of standard functions, the compiler can have special knowledge of these standard functions embedded in it. This special knowledge is used in various ways (open coding, inline expansion, source transformation), but the implications to the user are basically the same:

- Attempts to redefine standard functions may be frustrated, since the function may never be called. Although it is technically illegal to redefine standard functions, users sometimes want to implicitly redefine these functions when they are debugging using the `trace` macro. Special-casing of standard functions can be inhibited using the `notinline` declaration, but even then some phases of analysis such as type inferencing are applied by the compiler.
- The compiler can have multiple alternate implementations of standard functions that implement different trade-offs of speed, space and safety. This selection is based on the compiler policy, [Compiler Policy](#).

When a function call is *open coded*, inline code whose effect is equivalent to the function call is substituted for that function call. When a function call is *closed coded*, it is usually left as is, although it might be turned into a call to a different function with different arguments. As an example, if `nthcdr` were to be open coded, then

```
(nthcdr 4 foobar)
```

might turn into

```
(cdr (cdr (cdr (cdr foobar))))
```

or even

```
(do ((i 0 (1+ i))
      (list foobar (cdr foobar)))
      (= i 4) list))
```

If `nth` is closed coded, then

```
(nth x l)
```

might stay the same, or turn into something like

```
(car (nthcdr x l))
```

In general, open coding sacrifices space for speed, but some functions (such as `car`) are so simple that they are always open-coded. Even when not open-coded, a call to a standard function may be transformed into a different function call (as in the last example) or compiled as *static call*. Static function call uses a more efficient calling convention that forbids redefinition.

4.6 Interpreter

By default SBCL implements `eval` by calling the native code compiler.

SBCL also includes an interpreter for use in special cases where using the compiler is undesirable, for example due to compilation overhead. Unlike in some other Lisp implementations, in SBCL interpreted code is not safer or more debuggable than compiled code.

- **[variable]** `sb-ext:*evaluator-mode*` *:compile*

Toggle between different evaluator implementations. If set to `:compile`, an implementation of `eval` that calls the compiler will be used. If set to `:interpret`, an interpreter will be used.

4.7 Advanced Compiler Use and Efficiency Hints

For more advanced usages of the compiler, please see the chapter of the same name in the CMUCL manual. Many aspects of the compiler have stayed exactly the same, and there is a much more detailed explanation of the compiler's behavior and how to maximally optimize code in their manual. In particular, while SBCL no longer supports byte-code compilation, it does support CMUCL's block compilation facility allowing whole program optimization and increased use of the local call convention.

Unlike CMUCL, SBCL is able to open-code forward-referenced type tests while block compiling. This helps for mutually referential `defstructs` in particular.

5 Debugger

This chapter documents the debugging facilities of SBCL, including the debugger, single-stepper and `trace(0 1)`, and the effect of `(optimize debug)` declarations.

5.1 Debugger Entry

5.1.1 Debugger Banner

When you enter the debugger, it looks something like this:

```
debugger invoked on a TYPE-ERROR in thread 11184:
  The value 3 is not of type LIST.

You can type HELP for debugger help, or (SB-EXT:QUIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [ABORT   ] Reduce debugger level (leaving debugger, returning to toplevel).
  1: [TOPLEVEL] Restart at toplevel READ/EVAL/PRINT loop.
(CAR 1 3)
0]
```

The first group of lines describe what the error was that put us in the debugger. In this case `car` was called on 3, causing a `type-error`.

This is followed by the "beginner help line", which appears only if `sb-debug:*debug-beginner-help-p*` is true (default).

Next comes a listing of the active restart names, along with their descriptions -- the ways we can restart execution after this error. In this case, both options return to top-level. Restarts can be selected by entering the corresponding number or name.

The current frame appears right underneath the restarts, immediately followed by the debugger prompt.

5.1.2 Debugger Invocation

The debugger is invoked when:

- `error(0 1)` is called, and the condition it signals is not handled.
- `break` is called, or `signal` is called with a condition that matches the current `*break-on-signals*`.
- The debugger is explicitly entered with the `invoke-debugger` function.

When the debugger is invoked by a condition, ANSI mandates that the value of `*debugger-hook*`, if any, be called with two arguments: the condition that caused the debugger to be invoked and the previous value of `*debugger-hook*`. When this happens, `*debugger-hook*` is bound to `nil` to prevent recursive errors. However, ANSI also mandates that `*debugger-hook*` not be invoked when the debugger is to be entered by the `break` function. For users who wish to provide an alternate debugger interface (and thus catch `break` entries into the debugger),

SBCL provides `sb-ext:*invoke-debugger-hook*`, which is invoked during any entry into the debugger.

- **[variable]** `sb-ext:*invoke-debugger-hook*` `#<function swank/sbcl::swank-invoke-debugger-hook {1201C2C64B}>`

This is either `nil` or a designator for a function of two arguments, to be run when the debugger is about to be entered. The function is run with `*invoke-debugger-hook*` bound to `nil` to minimize recursive errors, and receives as arguments the condition that triggered debugger entry and the previous value of `*invoke-debugger-hook*`.

This mechanism is an SBCL extension similar to the standard `*debugger-hook*`. In contrast to `*debugger-hook*`, it is observed by `invoke-debugger` even when called by `break`.

5.2 Debugger Command Loop

The debugger is an interactive read-eval-print loop much like the normal top level, but some symbols are interpreted as debugger commands instead of being evaluated. A debugger command starts with the symbol name of the command, possibly followed by some arguments on the same line. Some commands prompt for additional input. Debugger commands can be abbreviated by any unambiguous prefix: `help` can be typed as `h`, `he`, etc.

The package is not significant in debugger commands; any symbol with the name of a debugger command will work. If you want to show the value of a variable that happens also to be the name of a debugger command you can wrap the variable in a `progn` to hide it from the command loop.

The debugger prompt is `<frame>]`, where `<frame>` is the number of the current frame. Frames are numbered starting from zero at the top (most recent call), increasing down to the bottom. The current frame is the frame that commands refer to.

It is possible to override the normal printing behaviour in the debugger by using the `sb-ext:*debug-print-variable-alist*`.

- **[variable]** `sb-ext:*debug-print-variable-alist*` `nil`

an association list describing new bindings for special variables to be used within the debugger. Eg.

```
((*print-length* . 10) (*print-level* . 6) (*print-pretty* . nil))
```

The variables in the `car` positions are bound to the values in the `cdr` during the execution of some debug commands. When evaluating arbitrary expressions in the debugger, the normal values of the printer control variables are in effect.

Initially empty, `DEBUG-PRINT-VARIABLE-ALIST` is typically used to provide bindings for printer control variables.

5.3 Stack Frames

A *stack frame* is the run-time representation of a call to a function; the frame stores the state that a function needs to remember what it is doing. Frames have:

- *Variables* (see [Variable Access](#)), which are the values being operated on.
- *Arguments* to the call (which are really just particularly interesting variables).
- A current source location ([Source Location Printing](#)), which is the place in the program where the function was running when it stopped to call another function, or because of an interrupt or error.

5.3.1 Stack Motion

These commands move to a new stack frame and print the name of the function and the values of its arguments in the style of a Lisp function call:

- *up*: Move up to the next higher frame. More recent function calls are considered to be higher on the stack.
- *down*: Move down to the next lower frame.
- *top*: Move to the highest frame, that is, the frame where the debugger was entered.
- *bottom*: Move to the lowest frame.
- *frame* [*<n>*]: Move to the frame with the specified number. Prompts for the number if not supplied. The frame with number 0 is the frame where the debugger was entered.

5.3.2 How Arguments are Printed

A frame is printed to look like a function call, but with the actual argument values in the argument positions. So the frame for this call in the source:

```
(myfun (+ 3 4) 'a)
```

would look like this:

```
(MYFUN 7 A)
```

All keyword and optional arguments are displayed with their actual values; if the corresponding argument was not supplied, the value will be the default. So this call:

```
(subseq "foo" 1)
```

would look like this:

```
(SUBSEQ "foo" 1 3)
```

And this call:

```
(string-upcase "test case")
```

would look like this:

```
(STRING-UPCASE "test case" :START 0 :END NIL)
```

The arguments to a function call are displayed by accessing the argument variables. Although those variables are initialized to the actual argument values, they can be set inside the function; in this case the new value will be displayed.

&rest arguments are handled somewhat differently. The value of the rest argument variable is displayed as the spread-out arguments to the call, so:

```
(format t "~A is a ~A." "This" 'test)
```

would look like this:

```
(FORMAT T "~A is a ~A." "This" 'TEST)
```

Rest arguments cause an exception to the normal display of keyword arguments in functions that have both **&rest** and **&key** arguments. In this case, the keyword argument variables are not displayed at all; the rest arg is displayed instead. So for these functions, only the keywords actually supplied will be shown, and the values displayed will be the argument values, not values of the (possibly modified) variables.

If the variable for an argument is never referenced by the function, it will be deleted. The variable value is then unavailable, so the debugger prints `#<unused-arg>` instead of the value. Similarly, if for any of a number of reasons the value of the variable is unavailable or not known to be available ([Variable Access](#)), then `#<unavailable-arg>` will be printed instead of the argument value.

Note that inline expansion and open-coding affect what frames are present in the debugger, see [Debugger Policy Control](#).

5.3.3 Function Names

If a function is defined by **defun** it will appear in backtrace by that name. Functions defined by **labels** and **flet** will appear as `(FLET <name>)` and `(LABELS <name>)` respectively. Anonymous lambdas will appear as `(LAMBDA <lambda-list>)`.

Entry Point Details Sometimes the compiler introduces new functions that are used to implement a user function, but are not directly specified in the source. This is mostly done for argument type and count checking.

With recursive or block compiled functions, an additional `external` frame may appear before the frame representing the first call to the recursive function or entry to the compiled block. This is a consequence of the way the compiler works: there is nothing odd with your program. You may also see `cleanup` frames during the execution of **unwind-protect** cleanup code, and `optional` for variable argument entry points.

5.3.4 Debug Tail Recursion

The compiler is *properly tail recursive*. If a function call is in a tail-recursive position, the stack frame will be deallocated *at the time of the call*, rather than after the call returns. Consider this backtrace:

```
(BAR ...)  
(FOO ...)
```

Because of tail recursion, it is not necessarily the case that `foo` directly called `bar`. It may be that `foo` called some other function `foo2`, which then called `bar` tail-recursively, as in this example:

```
(defun foo ()  
  ...  
  (foo2 ...)  
  ...)  
  
(defun foo2 (...)  
  ...  
  (bar ...))  
  
(defun bar (...)  
  ...)
```

Usually the elimination of tail-recursive frames makes debugging more pleasant, since these frames are mostly uninformative. If there is any doubt about how one function called another, it can usually be eliminated by finding the source location in the calling frame. See [Source Location Printing](#).

The elimination of tail-recursive frames can be prevented by disabling tail-recursion optimization, which happens when the `debug` optimization quality is greater than 2. See [Debugger Policy Control](#).

5.3.5 Unknown Locations and Interrupts

The debugger operates using special debugging information attached to the compiled code. This debug information tells the debugger what it needs to know about the locations in the code where the debugger can be invoked. If the debugger somehow encounters a location not described in the debug information, then it is said to be *unknown*. If the code location for a frame is unknown, then some variables may be inaccessible, and the source location cannot be precisely displayed.

There are three reasons why a code location could be unknown:

- There is inadequate debug information due to the value of the `debug` optimization quality. See [Debugger Policy Control](#).
- The debugger was entered because of an interrupt such as C-c.
- A hardware error such as a bus error occurred in code that was compiled unsafely due to the value of the `safety` optimization quality.

In the last two cases, the values of argument variables are accessible, but may be incorrect. For more details on when variable values are accessible, see [Variable Value Availability](#).

It is possible for an interrupt to happen when a function call or return is in progress. The debugger may then flame out with some obscure error or insist that the bottom of the stack has

been reached, when the real problem is that the current stack frame can't be located. If this happens, return from the interrupt and try again.

5.4 Variable Access

There are two ways to access the current frame's local variables in the debugger: `list-locals` and `sb-debug:var`.

The debugger doesn't really understand lexical scoping; it has just one namespace for all the variables in the current stack frame. If a symbol is the name of multiple variables in the same function, then the reference appears ambiguous, even though lexical scoping specifies which value is visible at any given source location. If the scopes of the two variables are not nested, then the debugger can resolve the ambiguity by observing that only one variable is accessible.

When there are ambiguous variables, the evaluator assigns each one a small integer identifier. The `sb-debug:var` function uses this identifier to distinguish between ambiguous variables. The `list-locals` command prints the identifier. In the following example, there are two variables named `x`. The first one has identifier 0 (which is not printed), the second one has identifier 1.

```
x = 1
x#1 = 2
```

- `list-locals [<prefix>]`: This command prints the name and value of all variables in the current frame whose name has the specified `<prefix>`, which may be a string or a symbol. If no `<prefix>` is given, then all available variables are printed. If a variable has a potentially ambiguous name, then the name is printed with a `#<identifier>` suffix, where `<identifier>` is the small integer used to make the name unique.
- **[function]** `sb-debug:var` *name &optional (id 0 sb-debug::id-supplied)*

Return a variable's value if possible. `name` is a simple-string or symbol. If it is a simple-string, it is an initial substring of the variable's name. If `name` is a symbol, it has the same name and package as the variable whose value this function returns. If the symbol is uninterned, then the variable has the same name as the symbol, but it has no package.

If `name` is the initial substring of variables with different names, then this returns no values after displaying the ambiguous names. If `name` determines multiple variables with the same name, then you must use the optional ID argument to specify which one you want. If you left ID unspecified, then this returns no values after displaying the distinguishing id values.

The result of this function is limited to the availability of variable information. This is `setf`-able.

5.4.1 Variable Value Availability

The value of a variable may be unavailable to the debugger in portions of the program where Lisp says that the variable is defined. If a variable value is not available, the debugger will not let you read or write that variable. With one exception, the debugger will never display an incorrect value for a variable. Rather than displaying incorrect values, the debugger tells you the value is unavailable.

The one exception is this: if you interrupt (e.g. with C - c) or if there is an unexpected hardware error such as a bus error (which should only happen in unsafe code), then the values displayed for arguments to the interrupted frame might be incorrect. This exception applies only to the interrupted frame: any frame farther down the stack will be fine.

(Note: Since the location of an interrupt or hardware error will always be an unknown location, non-argument variable values will never be available in the interrupted frame. See [Unknown Locations and Interrupts](#).)

The value of a variable may be unavailable for these reasons:

- The value of the **debug** optimization quality may have omitted debug information needed to determine whether the variable is available. Unless a variable is an argument, its value will only be available when **debug** is at least 2.
- The compiler did lifetime analysis and determined that the value was no longer needed, even though its scope had not been exited. Lifetime analysis is inhibited when the **debug** optimization quality is 3.
- The variable's name is an uninterned symbol (gensym). To save space, the compiler only dumps debug information about uninterned variables when the **debug** optimization quality is 3.
- The frame's location is unknown (see [Unknown Locations and Interrupts](#)) because the debugger was entered due to an interrupt or unexpected hardware error. Under these conditions the values of arguments will be available, but might be incorrect. This is the exception mentioned above.
- The variable (or the code referencing it) was optimized out of existence. Variables with no reads are always optimized away. The degree to which the compiler deletes variables will depend on the value of the **compilation-speed** optimization quality, but most source-level optimizations are done under all compilation policies.
- The variable is never set and its definition looks like

```
(LET ((var1 var2))  
  ...)
```

In this case, `var1` is substituted with `var2`.

- The variable is never set and is referenced exactly once. In this case, the reference is substituted with the variable initial value.

Since it is especially useful to be able to get the arguments to a function, argument variables are treated specially when the **speed** optimization quality is less than 3 and the **debug** quality is at least 1. With this compilation policy, the values of argument variables are almost always available everywhere in the function, even at unknown locations. For non-argument variables, **debug** must be at least 2 for values to be available, and even then, values are only available at known locations.

5.4.2 Note On Lexical Variable Access

When the debugger command loop establishes variable bindings for available variables, these variable bindings have lexical scope and dynamic extent. You can close over them, but such closures can't be used as upward function arguments.

Note: The variable bindings are actually created using the Lisp `symbol-macrolet` special form.

You can also set local variables using `setq`, but if the variable was closed over in the original source and never set, then setting the variable in the debugger may not change the value in all the functions the variable is defined in. Another risk of setting variables is that you may assign a value of a type that the compiler proved the variable could never take on. This may result in bad things happening.

5.5 Source Location Printing

One of the debugger's capabilities is source level debugging of compiled code. These commands display the source location for the current frame:

- `source [<context>]`: This command displays the file that the current frame's function was defined from (if it was defined from a file), and then the source form responsible for generating the code that the current frame was executing. If `<context>` is specified, then it is an integer specifying the number of enclosing levels of list structure to print.

The source form for a location in the code is the innermost list present in the original source that encloses the form responsible for generating that code. If the actual source form is not a list, then some enclosing list will be printed. For example, if the source form was a reference to the variable `*some-random-special*`, then the innermost enclosing evaluated form will be printed. Here are some possible enclosing forms:

```
(let ((a *some-random-special*))
  ...)

(+ *some-random-special* ...)
```

If the code at a location was generated from the expansion of a macro or a source-level compiler optimization, then the form in the original source that expanded into that code will be printed. Suppose the file `/usr/me/mystuff.lisp` looked like this:

```
(defmacro mymac ()
  '(myfun))

(defun foo ()
  (mymac)
  ...)
```

If `foo` has called `myfun`, and is waiting for it to return, then the `source` command would print:

```
; File: /usr/me/mystuff.lisp

(MYMAC)
```

Note that the macro use was printed, not the actual function call form, (myfun).

If enclosing source is printed by giving an argument to `source` or `vsource`, then the actual source form is marked by wrapping it in a list whose first element is `#:***here***`. In the previous example, `source 1` would print:

```
; File: /usr/me/mystuff.lisp

(DEFUN F00 ()
  (#:***HERE***
   (MYMAC))
  ...)
```

5.5.1 How the Source is Found

If the code was defined from Lisp by `compile` or `eval`, then the source can always be reliably located. If the code was defined from a FASL file created by `compile-file`, then the debugger gets the source forms it prints by reading them from the original source file. This is a potential problem, since the source file might have moved or changed since the time it was compiled.

The source file is opened using the `truename` of the source file pathname originally given to the compiler. This is an absolute pathname with all logical names and symbolic links expanded. If the file can't be located using this name, then the debugger gives up and signals an error.

If the source file can be found, but has been modified since the time it was compiled, the debugger prints this warning:

```
; File has been modified since compilation:
;   <filename>
; Using form offset instead of character position.
```

where `<filename>` is the name of the source file. It then proceeds using a robust but not foolproof heuristic for locating the source. This heuristic works if:

- No top-level forms before the top-level form containing the source have been added or deleted, and
- the top-level form containing the source has not been modified much. (More precisely, none of the list forms beginning before the source form have been added or deleted.)

If the heuristic doesn't work, the displayed source will be wrong, but will probably be near the actual source. If the "shape" of the top-level form in the source file is too different from the original form, then an error will be signaled. When the heuristic is used, the source location commands are noticeably slowed.

Source location printing can also be confused if (after the source was compiled) a read-macro you used in the code was redefined to expand into something different, or if a read-macro ever returns the same `eq` list twice. If you don't define read macros and don't use `##` in perverted ways, you don't need to worry about this.

5.5.2 Source Location Availability

Source location information is only available when the `debug` optimization quality is at least 2. If source location information is unavailable, the source commands will give an error message.

If source location information is available, but the source location is unknown because of an interrupt or unexpected hardware error (see [Unknown Locations and Interrupts](#)), then the command will print

```
Unknown location: using block start.
```

and then proceed to print the source location for the start of the *basic block* enclosing the code location. It's a bit complicated to explain exactly what a basic block is, but here are some properties of the block start location:

- The block start location may be the same as the true location.
- The block start location will never be later in the program's flow of control than the true location.
- No conditional control structures (such as `if`, `cond`, or `(0 1)`) will intervene between the block start and the true location (but note that some conditionals present in the original source could be optimized away.) Function calls *do not* end basic blocks.
- The head of a loop will be the start of a block.
- The programming language concept of block structure and the Lisp `block` special form are totally unrelated to the compiler's basic block.

In other words, the true location lies between the printed location and the next conditional (but watch out because the compiler may have changed the program on you.)

5.6 Debugger Policy Control

The compilation policy specified by `optimize` declarations affects the behavior seen in the debugger. The `debug` quality directly affects the debugger by controlling the amount of debugger information dumped. Other optimization qualities have indirect but observable effects due to changes in the way compilation is done.

Unlike the other optimization qualities (which are compared in relative value to evaluate tradeoffs), the `debug` optimization quality is directly translated to a level of debug information. This absolute interpretation allows the user to count on a particular amount of debug information being available even when the values of the other qualities are changed during compilation. These are the levels of debug information that correspond to the values of the `debug` quality:

- 0: Only the function name and enough information to allow the stack to be parsed.
- > 0: Any level greater than 0 gives level 0 plus all argument variables. Values will only be accessible if the argument variable is never set and `speed` is not 3. SBCL allows any real value for optimization qualities. It may be useful to specify 0.5 to get backtrace argument display without argument documentation.

- 1: Level 1 provides argument documentation (printed argument lists) and derived argument/result type information. This makes `describe` more informative, and allows the compiler to do compile-time argument count and type checking for any calls compiled at run-time. This is the default.
- 2: Level 1 plus all interned local variables, source location information, and lifetime information that tells the debugger when arguments are available (even when `speed` is 3 or the argument is set).
- > 2: Any level greater than 2 gives level 2 and in addition disables tail-call optimization, so that the backtrace will contain frames for all invoked functions, even those in tail positions.
- 3: Level 2 plus all uninterned variables. In addition, lifetime analysis is disabled (even when `speed` is 3), ensuring that all variable values are available at any known location within the scope of the binding. This has a speed penalty in addition to the obvious space penalty.

Inlining of local functions is inhibited so that they may be `trace(0 1)d`.

- > (max speed space): If `debug` is greater than both `speed` and `space`, the command `return` can be used to continue execution by returning a value from the current stack frame.
- > (max speed space compilation-speed): If `debug` is greater than all of `speed`, `space` and `compilation-speed` the code will be steppable (see [Single Stepping](#)).

As you can see, if the `speed` quality is 3, debugger performance is degraded. This effect comes from the elimination of argument variable special-casing (see [Variable Value Availability](#)). Some degree of speed/debuggability tradeoff is unavoidable, but the effect is not too drastic when `debug` is at least 2.

In addition to `inline` and `notinline` declarations, the relative values of the `speed` and `space` qualities also change whether functions are inline expanded. If a function is inline expanded, then there will be no frame to represent the call, and the arguments will be treated like any other local variable. Functions may also be *semi-inline*, in which case there is a frame to represent the call, but the call is to an optimized local version of the function, not to the original function.

5.7 Exiting Commands

These commands get you out of the debugger.

- `toplevel`: Throw to top level.
- `restart [<n>]`: Invoke the <n>th restart case as displayed by the `error(0 1)` command. If <n> is not specified, the available restart cases are reported.
- `continue`: Call `continue(0 1)` on the condition given to `debug`. If there is no restart case named `continue`, then an error is signaled.
- `abort`: Call `abort(0 1)` on the condition given to `debug`. This is useful for popping debug command loop levels or aborting to top level, as the case may be.

- `return <value>`: Return `value` from the current stack frame. This command is available when the debug optimization quality is greater than both `speed` and `space`. Care must be taken that the value is of the same type as SBCL expects the stack frame to return.
- `restart-frame`: Restart execution of the current stack frame. This command is available when the debug optimization quality is greater than both `speed` and `space` and when the frame is for a global function. If the function is redefined in the debugger before the frame is restarted, the new function will be used.

5.8 Information Commands

Most of these commands print information about the current frame or function, but a few show general information.

- `help` or `?`: Display a synopsis of debugger commands.
- `describe`: Call `describe` on the current function and displays the number of local variables.
- `print`: Display the current function call as it would be displayed by moving to this frame.
- `error`: Print the condition given to `invoke-debugger` and the active proceed cases.
- `backtrace [<n>]`: Display all the frames from the current to the bottom. Only shows `<n>` frames if specified. The printing is controlled by `sb-debug:*debug-print-variable-alist*`.

5.9 Breakpoint Commands

SBCL supports setting of breakpoints inside compiled functions and stepping of compiled code. Breakpoints can only be set at known locations (see [Unknown Locations and Interrupts](#)), so these commands are largely useless unless the `debug` optimize quality is at least 2 (see [Debugger Policy Control](#)). These commands manipulate breakpoints:

- `breakpoint <location> [<option> <value>]*`: Set a breakpoint in some function. `<location>` may be an integer code location number (as displayed by `list-locations`) or a keyword. The keyword can be used to indicate setting a breakpoint at the function start (`:start`, `:s`) or function end (`:end`, `:e`). The breakpoint command has `:condition`, `:break`, `:print` and `:function` options which work similarly to the `trace(0 1)` options.
- `list-locations [<function>]` or `ll [<function>]`: List all the code locations in the current frame's function, or in `<function>` if it is supplied. The display format is the code location number, a colon and then the source form for that location:

```
3: (1- N)
```

If consecutive locations have the same source, then a numeric range like `3-5:` will be printed. For example, a default function call has a known location both immediately before and after the call, which would result in two code locations with the same source. The listed function becomes the new default function for breakpoint setting (via the `breakpoint`) command.

- `list-breakpoints` or `lb`: List all currently active breakpoints with their breakpoint number.
- `delete-breakpoint [<number>]` or `db [<number>]`: Delete a breakpoint specified by its breakpoint number. If no number is specified, delete all breakpoints.
- `step*`: Step to the next possible breakpoint location in the current function. This always steps over function calls, instead of stepping into them.

5.9.1 Breakpoint Example

Consider this definition of the factorial function:

```
(defun ! (n)
  (if (zerop n)
      1
      (* n (! (1- n)))))
```

This debugger session demonstrates the use of breakpoints:

```
* (break) ; invoke debugger

debugger invoked on a SIMPLE-CONDITION in thread 11184: break

restarts (invokable by number or by possibly-abbreviated name):
  0: [CONTINUE] Return from BREAK.
  1: [ABORT   ] Reduce debugger level (leaving debugger, returning to toplevel).
  2: [TOPLEVEL] Restart at toplevel READ/EVAL/PRINT loop.
("varargs entry for top level local call BREAK" "break")
0] ll #'!

0-1: (SB-INT:NAMED-LAMBDA ! (N) (BLOCK ! (IF (ZEROP N) 1 (* N (! #)))))
2: (BLOCK ! (IF (ZEROP N) 1 (* N (! (1- N)))))
3: (ZEROP N)
4: (* N (! (1- N)))
5: (1- N)
6: (! (1- N))
7-8: (* N (! (1- N)))
9-10: (IF (ZEROP N) 1 (* N (! (1- N))))
0] br 4

(* N (! (1- N)))
1: 4 in !
added
0] toplevel

> (! 10) ; Call the function

*Breakpoint hit*

Restarts:
  0: [CONTINUE] Return from BREAK.
  1: [ABORT   ] Return to Top-Level.
```

```

Debug (type H for help)

(! 10) ; We are now in first call (arg 10) before the multiply
Source: (* N (! (1- N)))
3] step*

*Step*

(! 10) ; We have finished evaluation of (1- n)
Source: (1- N)
3] step*

*Breakpoint hit*

Restarts:
  0: [CONTINUE] Return from BREAK.
  1: [ABORT    ] Return to Top-Level.

Debug (type H for help)

(! 9) ; We hit the breakpoint in the recursive call
Source: (* N (! (1- N)))
3]

```

Note: The `step*` command differs from the single stepping commands in that it also functions in compiled code which has not been compiled with stepping instrumentation. It simply steps to the next compiled code location. In the future, this form of stepping may be improved enough to subsume the instrumentation based stepping commands, which have much higher overhead.

5.10 Function Tracing

The tracer causes selected functions to print their arguments and their results whenever they are called. Options allow conditional printing of the trace information and conditional breakpoints on function entry or exit.

In SBCL, tracing can be done either by temporarily redefining the function name (encapsulation), or using breakpoints. When breakpoints are used, the function object itself is destructively modified to cause the tracing action. The advantage of using breakpoints is that tracing works even when the function is anonymously called via `funcall`, that function object identity is preserved, and that anonymous and local functions can also be traced.

- **[macro]** `trace` *&rest specs*

```
trace {Option Global-Value}* {Name {Option Value}*}*

```

`trace` is a debugging tool that provides information when specified functions are called. In its simplest form:

```
(TRACE NAME-1 NAME-2 ...)
```

The names are not evaluated. Each may be one of the following: `* symbol`, denoting a function or macro. `* fname`, a valid function name, denoting a function. `* (method`

`fname qualifiers* (specializers*)` denoting a method. `*` (compiler-macro symbol) denoting a compiler macro. `*` (labels fname :in outer-name) or (flet fname :in outer-name) denoting a local function where outer-name may be any of the previous names for functions, macros, methods or compiler macros. Tracing local functions may require `debug` policy 3 to inhibit inlining. `* string(0 1)` denoting all functions fbound to symbols whose home package is the package with the given name.

Options allow modification of the default behavior. Each option is a pair of an option keyword and a value form. Global options are specified before the first name, and affect all functions traced by a given use of `trace`. Options may also be interspersed with function names, in which case they act as local options, only affecting tracing of the immediately preceding function name. Local options override global options.

By default, `trace` causes a printout on `*trace-output*` each time that one of the named functions is entered or returns. (This is the basic, ANSI Common Lisp behavior of `trace`.)

The following options are defined:

- `:report <report-type>`

If `report-type` is `trace` (the default) then information is reported by printing immediately. If `report-type` is `nil`, then the only effect of the trace is to execute other options (e.g. `print` or `break`). Otherwise, `report-type` is treated as a function designator and, for each trace event, funcalled with 5 arguments: trace depth (a non-negative integer), a function name or a function object, a keyword (`:enter`, `:exit` or `:non-local-exit`), a stack frame, and a list of values (arguments or return values).

- `:condition <form>`
- `:condition-after <form>`
- `:condition-all <form>`

If `:condition` is specified, then `trace` does nothing unless `form` evaluates to true at the time of the call. `:condition-after` is similar, but suppresses the initial printout, and is tested when the function returns. `:condition-all` tries both before and after.

- `:break <form>`
- `:break-after <form>`
- `:break-all <form>`

If specified, and `form` evaluates to true, then the debugger is invoked at the start of the function, at the end of the function, or both, according to the respective option.

- `:print <form>`
- `:print-after <form>`
- `:print-all <form>`

In addition to the usual printout, the result of evaluating `form` is printed at the start of the function, at the end of the function, or both, according to the respective option.

Multiple print options cause multiple values to be printed.

- `:wherein <names>`

If specified, `names` is a function name or list of names. `trace` does nothing unless a call to one of those functions encloses the call to this function (i.e. it would appear in a backtrace.) Anonymous functions have string names like "`defun FOO`".

- `:encapsulate { :default | t | nil }`

If `t`, the default, tracing is done via encapsulation (redefining the function name) rather than by modifying the function. `:default` is not the default but means to use encapsulation for interpreted functions and funcallable instances, breakpoints otherwise. When encapsulation is used, forms are *not* evaluated in the function's lexical environment, but `sb-debug:arg` can still be used.

- `:methods { t | nil }`

If `t`, any function argument naming a generic function will have its methods traced in addition to the generic function itself.

- `:function <function-form>`

This is not really an option but rather another way of specifying what function to trace. The `function-form` is evaluated immediately, and the resulting function is traced.

`:condition`, `:break` and `:print` forms are evaluated in a context which mocks up the lexical environment of the called function, so that `sb-debug:var` and `sb-debug:arg` can be used. The `-AFTER` and `-ALL` forms can also use `sb-debug:arg`. In forms which are evaluated after the function call, `(sb-debug:arg N)` returns the N-th value returned by the function.

In the case of functions where the known return convention is used to optimize, encapsulation may be necessary in order to make tracing work at all. The symptom of this occurring is an error stating

```
Error in function F00: :FUNCTION-END breakpoints are
currently unsupported for the known return convention.
```

in such cases we recommend using `(TRACE F00 :ENCAPSULATE t)`.

- **[macro]** `untrace` *&rest specs*

Remove tracing from the specified functions. Untraces all functions when called with no arguments.

- **[variable]** `sb-debug:*trace-indentation-step*` 2

The increase in trace indentation at each call level.

- **[variable]** `sb-debug:*max-trace-indentation*` 40

If the trace indentation exceeds this value, then indentation restarts at 0.

- [variable] `sb-debug:*trace-encapsulate-default*` *t*

The default value for the `:encapsulate` option to `trace(0 1)`.

- [variable] `sb-debug:*trace-report-default*` *trace*

The default value for the `:report` option to `trace(0 1)`.

5.11 Single Stepping

SBCL includes an instrumentation based single-stepper for compiled code, that can be invoked via the `step` macro, or from within the debugger. See [Debugger Policy Control](#), for details on enabling stepping for compiled code.

The following debugger commands are used for controlling single stepping.

- `start`: Select the `continue` restart if one exists and starts single stepping. None of the other single stepping commands can be used before stepping has been started either by using `start` or by using the standard `step` macro.
- `step`: Step into the current form. Stepping will be resumed when the next form that has been compiled with stepper instrumentation is evaluated.
- `next`: Step over the current form. Stepping will be disabled until evaluation of the form is complete.
- `out`: Step out of the current frame. Stepping will be disabled until the topmost stack frame that had been stepped into returns.
- `stop`: Stop the single stepper and resumes normal execution.
- [macro] `step` *form*

The form is evaluated with single stepping enabled. Function calls outside the lexical scope of the form can be stepped into only if the functions in question have been compiled with sufficient `debug` policy to be at least partially steppable.

5.12 Enabling and Disabling the Debugger

In certain contexts (e.g. non-interactive applications), it may be desirable to turn off the SBCL debugger (and possibly re-enable it). The functions here control the debugger.

- [function] `sb-ext:disable-debugger`

When invoked, this function will turn off both the SBCL debugger and `ldb` (the low-level debugger). See also [enable-debugger](#).

- [function] `sb-ext:enable-debugger`

Restore the debugger if it has been turned off by [disable-debugger](#).

6 Efficiency

6.1 Slot access

6.1.1 Structure object slot access

Structure slot accessors are efficient only if the compiler is able to open code them: compiling a call to a structure slot accessor before the structure is defined, declaring one `notinline`, or passing it as a functional argument to another function causes severe performance degradation.

6.1.2 Standard object slot access

The most efficient way to access a slot of a `standard-object` is by using `slot-value` with a constant slot name argument inside a `defmethod` body, where the variable holding the instance is a `specializer` parameter of the method and is never assigned to. The cost is roughly 1.6 times that of an open coded structure slot accessor.

Second most efficient way is to use a CLOS slot accessor, or `slot-value` with a constant slot name argument, but in circumstances other than specified above. This may be up to 3 times as slow as the method described above.

Example:

```
(defclass foo () ((bar)))

;; Fast: specializer and never assigned to
(defmethod quux ((foo foo) new)
  (let ((old (slot-value foo 'bar)))
    (setf (slot-value foo 'bar) new)
    old))

;; Slow: not a specializer
(defmethod quux ((foo foo) new)
  (let* ((temp foo)
        (old (slot-value temp 'bar)))
    (setf (slot-value temp 'bar) new)
    old))

;; Slow: assignment to F00
(defmethod quux ((foo foo) new)
  (let ((old (slot-value foo 'bar)))
    (setf (slot-value foo 'bar) new)
    (setf foo new)
    old))
```

Note that when profiling code such as this, the first few calls to the generic function are not representative, as the dispatch mechanism is lazily set up during those calls.

6.2 Stack allocation

SBCL has fairly extensive support for performing allocations on the stack when a variable or function is declared `dynamic-extent`. The `dynamic-extent` declarations are not verified but

are simply trusted as long as `sb-ext:*stack-allocate-dynamic-extent*` is true.

- **[variable]** `sb-ext:*stack-allocate-dynamic-extent*` *t*

If true (the default), the compiler believes `dynamic-extent` declarations and stack allocates otherwise inaccessible parts of the object whenever possible.

SBCL recognizes any value which a variable declared `dynamic-extent` can take on as having dynamic extent. This means that, in addition to the value a variable is bound to initially, a value assigned to a variable by `setq` is also recognized as having dynamic extent when the variable is declared `dynamic-extent`. Users can thus build complex structures on the stack using iteration and `setq`.

At present, SBCL implements stack allocation for the following kinds of values when they are recognized as having dynamic extent:

- `&rest` lists;
- the results of `cons(0 1)`, `list(0 1)`, `list*`, and `vector(0 1)`;
- the result of simple forms of `make-array`: stack allocation is possible only if the resulting array is known to be both simple and one-dimensional, and has a constant `:element-type`;

Note: Stack space is limited, so allocation of a large vector may cause stack overflow. Stack overflow checks are done except in zero `safety` policies.

- closures defined with `flet` or `labels` with a bound `dynamic-extent` declaration;
- anonymous closures defined with `lambda(0 1)`;
- user-defined structures when the structure constructor defined using `defstruct` has been declared `inline`;

Note: Structures with `raw` slots can currently be stack-allocated only on x86 and x86-64. A raw slot is one whose declared type is a subtype of exactly one of: `double-float`, `single-float`, `(complex double-float)`, `(complex single-float)`, or `sb-ext:word`; but as an exception to the preceding, any subtype of `fixnum` is not stored as raw despite also being a subtype of `sb-ext:word`.

- otherwise-inaccessible parts of objects recognized to be dynamic extent. The support for detecting when this applies is very sophisticated. The compiler can do this detection when any value form for a variable contains conditional allocations, function calls, inlined functions, anonymous closures, or even other variables. This allows stack allocation of complex structures.

Examples:

```
;;; Declaiming a structure constructor inline before definition makes
;;; stack allocation possible.
(declaim (inline make-thing))
(defstruct thing obj next)

;;; Stack allocation of various objects bound to DYNAMIC-EXTENT
;;; variables.
(let* ((list (list 1 2 3))
```

```

    (nested (cons (list 1 2) (list* 3 4 (list 5))))
    (vector (make-array 3 :element-type 'single-float))
    (thing (make-thing :obj list
                      :next (make-thing :obj (make-array 3))))
    (closure (let ((y ...)) (lambda () y))))
    (declare (dynamic-extent list nested vector thing closure))
    ...)

;;; Stack allocation of objects assigned to DYNAMIC-EXTENT variables.
(let ((x nil))
  (declare (dynamic-extent x))
  (setq x (list 1 2 3))
  (dotimes (i 10)
    (setq x (cons i x)))
  ...)

;;; Stack allocation of arguments to a local function is equivalent
;;; to stack allocation of local variable values.
(flet ((f (x)
         (declare (dynamic-extent x))
         ...))
  ...
  (f (list 1 2 3))
  (f (cons (cons 1 2) (cons 3 4))))
  ...)

;;; Stack allocation of &REST lists
(defun foo (&rest args)
  (declare (dynamic-extent args))
  ...)

```

As a notable exception to recognizing otherwise inaccessible parts of other recognized dynamic extent values, SBCL does not as of 1.0.48.21 propagate dynamic-extentness through **&rest** arguments -- but another conforming implementation might, so portable code should not rely on this.

```

(declaim (inline foo))
(defun foo (fun &rest arguments)
  (declare (dynamic-extent arguments))
  (apply fun arguments))

(defun bar (a)
  ;; SBCL will heap allocate the result of (LIST A), and stack
  ;; allocate only the spine of the &rest list -- so this is
  ;; safe but unportable.
  ;;
  ;; Another implementation, including earlier versions of SBCL
  ;; might consider (LIST A) to be otherwise inaccessible and
  ;; stack-allocate it as well!
  (foo #'car (list a)))

```

If dynamic extent constraints specified in the Common Lisp standard are violated, the best that can happen is for the program to have garbage in variables and return values; more commonly,

the system will crash.

In particular, it is important to realize that this can interact in surprising ways with the otherwise inaccessible parts criterion:

```
(let* ((a (list 1 2 3))
      (b (cons a a)))
  (declare (dynamic-extent b))
  ;; Unless A is accessed elsewhere as well, SBCL will consider
  ;; it to be otherwise inaccessible -- it can only be accessed
  ;; through B, after all -- and stack allocate it as well.
  ;;
  ;; Hence returning (CAR B) here is unsafe.
  ...)
```

SBCL also performs sophisticated escape analysis to enable automatic stack allocation of local functions without any bound dynamic extent declarations in many situations where the compiler can prove that no uses escape (traditional Lisp terminology names this situation ‘all uses are downward funargs’). For example, in the following function, the local function `#'PREDICATEP` is stack allocated, because the compiler understands that the built-in function `position-if` only uses its first argument as a downward funarg:

```
(let ((acc 0))
  (flet ((predicatep (num) (plusp (+ num off)))))
    (dotimes (i 10)
      (incf acc (position-if #'predicatep array)))
    (if (plusp off)
        (incf acc (if (positivep acc) 10 3))
        (incf acc (position-if #'predicatep array))))
  acc)
```

Users can also declare that their own functions take downward funargs by adding bound dynamic extent declarations on the function arguments.

```
(defun trivial-hof (fun arg)
  (declare (dynamic-extent fun))
  (funcall fun 3 arg))
```

Currently, such dynamic extent declarations only cause stack allocation of downward funargs at call sites on sufficiently unsafe policy. This is partly because the compiler is currently not able to detect incorrect usage of dynamic extent declarations.

```
(defun autodxclosure1 (&optional (x 4))
  ;; Calling a higher-order function will only implicitly
  ;; stack-allocate a funarg if the callee is trusted (a CL:
  ;; function) or the caller is unsafe.
  (declare (optimize speed (safety 0) (debug 0)))
  (trivial-hof (lambda (a b) (+ a b x)) 92))
```

6.3 Modular arithmetic

Some numeric functions have a property: n lower bits of the result depend only on n lower bits of (all or some) arguments. If the compiler sees an expression of form `(LOGAND <expr> <mask>)`, where `<expr>` is a tree of such *good* functions and `<mask>` is known to be of type `(UNSIGNED-BYTE <w>)`, where `<w>` is a *good* width, all intermediate results will be cut to `<w>` bits (but it is not done for variables and constants!). This often results in an ability to use simple machine instructions for the functions.

Consider this example:

```
(defun i (x y)
  (declare (type (unsigned-byte 32) x y))
  (ldb (byte 32 0) (logxor x (lognot y))))
```

The result of `(lognot y)` will be negative and of type `(signed-byte 33)`, so a naive implementation on a 32-bit platform is unable to use 32-bit arithmetic here. But modular arithmetic optimizer is able to do it: because the result is cut down to 32 bits, the compiler will replace `logxor` and `lognot` with versions cutting results to 32 bits, and because terminals (here, expressions `x` and `y`) are also of type `(unsigned-byte 32)`, 32-bit machine arithmetic can be used.

As of SBCL 0.8.5 good functions are `+(0 1)`, `-(0 1)`, `logand`, `logior`, `logxor`, `lognot` and their combinations; and `ash` with the positive second argument. Good widths are 32 on 32-bit CPUs and 64 on 64-bit CPUs. While it is possible to support smaller widths as well, currently this is not implemented.

6.3.1 Signed modular arithmetic

Sign-extending the result in the following way will be translated into signed modular arithmetic:

```
(defun add (a b)
  (declare (type (signed-byte 64) a b))
  (let ((u (ldb (byte 64 0) (+ a b))))
    (logior u (- (mask-field (byte 1 63) u)))))
```

6.4 Recognized idioms

Common Lisp doesn't directly expose all features present in modern hardware. Some code patterns are recognized and turned into more efficient hardware instructions without requiring the use of internal features.

6.4.1 Count trailing zeros

```
(defun ctz (n)
  (declare (type (unsigned-byte 64) n))
  (integer-length (ldb (byte 64 0) (lognor n (- n)))))
```

is turned into hardware instructions on arm64 and x86-64. It returns 64 when `n` is 0. `n` can also be `(signed-byte 64)` or `fixnum`.

6.5 Global and Always-Bound variables

- **[macro]** `sb-ext:defglobal` *name value &optional (doc nil)*

Defines *name* as a global variable that is always bound. *value* is evaluated and assigned to *name* both at compile- and load-time, but only if *name* is not already bound.

Global variables share their values between all threads, and cannot be locally bound, declared special, defined as constants, and neither bound nor defined as symbol macros.

See also the declarations `sb-ext:global` and `sb-ext:always-bound`.

- **[declaration]** `sb-ext:global` *&REST SYMBOLS*

Only valid as a global proclamation.

Specifies that the named symbols cannot be proclaimed or locally declared **special**. Proclaiming an already special or constant variable name as `sb-ext:global` signal an error. Allows more efficient value lookup in threaded environments in addition to expressing programmer intention.

- **[declaration]** `sb-ext:always-bound` *&REST SYMBOLS*

Only valid as a global proclamation.

Specifies that the named symbols are always bound. Inhibits **makunbound** of the named symbols. Proclaiming an unbound symbol as `sb-ext:always-bound` signals an error. Allows the compiler to elide boundness checks from value lookups.

6.6 Miscellaneous Efficiency Issues

FIXME: The material in the CMUCL manual about getting good performance from the compiler should be reviewed, reformatted in Texinfo, lightly edited for SBCL, and substituted into this manual. In the meantime, the original CMUCL manual is still 95+% correct for the SBCL version of the Python compiler. See the sections

- Advanced Compiler Use and Efficiency Hints
- Advanced Compiler Introduction
- More About Types in Python
- Type Inference
- Source Optimization
- Tail Recursion
- Local Call
- Block Compilation
- Inline Expansion
- Object Representation
- Numbers

- General Efficiency Hints
- Efficiency Notes

Besides this information from the CMUCL manual, there are a few other points to keep in mind.

- The CMUCL manual doesn't seem to state it explicitly, but Python has a mental block about type inference when assignment is involved. Python is very aggressive and clever about inferring the types of values bound with `let`, `let*`, inline function call, and so forth. However, it's much more passive and dumb about inferring the types of values assigned with `setq`, `setf`, and friends. It would be nice to fix this, but in the meantime don't expect that just because it's very smart about types in most respects it will be smart about types involved in assignments. (This doesn't affect its ability to benefit from explicit type declarations involving the assigned variables, only its ability to get by without explicit type declarations.)
- Since the time the CMUCL manual was written, CMUCL (and thus SBCL) has gotten a generational garbage collector. This means that there are some efficiency implications of various patterns of memory usage which aren't discussed in the CMUCL manual. (Some new material should be written about this.)
- SBCL has some important known efficiency problems. Perhaps the most important are
 - The garbage collector is not particularly efficient, at least on platforms without the generational collector (as of SBCL 0.8.9, all except x86).
 - Various aspects of the PCL implementation of CLOS are more inefficient than necessary.

Finally, note that Common Lisp defines many constructs which, in the infamous phrase, "could be compiled efficiently by a sufficiently smart compiler". The phrase is infamous because making a compiler which actually is sufficiently smart to find all these optimizations systematically is well beyond the state of the art of current compiler technology. Instead, they're optimized on a case-by-case basis by hand-written code, or not optimized at all if the appropriate case hasn't been hand-coded. Some cases where no such hand-coding has been done as of SBCL version 0.6.3 include

- `(reduce #'f x)` where the type of `x` is known at compile time,
- various bit vector operations, e.g. `(position 0 some-bit-vector)`,
- specialized sequence idioms, e.g. `(remove item list :count 1)`,
- cases where local compilation policy does not require excessive type checking, e.g. `(locally (declare (safety 1)) (assoc item list))` (which currently performs safe `endp` checking internal to `assoc`).

If your system's performance is suffering because of some construct which could in principle be compiled efficiently, but which the SBCL compiler can't in practice compile efficiently, consider writing a patch to the compiler and submitting it for inclusion in the main sources. Such code is often reasonably straightforward to write; search the sources for the string `deftransform` to find many examples (some straightforward, some less so).